Video Frame Buffer Read and Video Frame Buffer Write v3.0

LogiCORE IP Product Guide

Vivado Design Suite

PG278 (v3.0) November 20, 2025

AMD

Table of Contents

Introduction

The AMD LogiCORE™ IP Video Frame Buffer Read and Video Frame Buffer Write cores provide high-bandwidth direct memory access between memory and AXI4-Stream video type target peripherals, which support the AXI4-Stream Video protocol.

Features

- AXI4 Compliant
- Streaming Video Format support for: RGB, RGBA, YUV 4:4:4, YUVA 4:4:4, YUV 4:2:2, YUV 4:2:0
- Provides programmable memory video format
- Supports memory video format in raster and specific tile modes
- Raster Mode Memory Video Format support for: RGBX8, BGRX8, YUVX8, YUYV8, UYVY8, RGBA8, BGRA8, YUVA8, RGBX10, YUVX10, Y_UV8, Y_UV8_420, RGB8, BGR8, YUV8, Y_UV10, Y_U_V8, Y_U_V10, Y_U_V12, Y_U_V8_420, Y_UV10_420, Y8, Y10, Y12
- Tile Mode Memory Video Format support for: Y_U_V8, Y_U_V10, Y_U_V12, Y_UV8, Y_UV10, Y_UV12, Y_UV8_420, Y_UV10_420, _UV12_420, Y8, Y10, Y12
- Supports progressive and interlaced video in Raster mode
- Supports only progressive video format in tile mode
- Raster Mode supports 8, 10, 12, and 16-bit per color component on stream interface and memory interface
- Tile Mode supports 8, 10, and 12-bit per color component on stream and memory interface
- Supports spatial resolutions from 64 × 64 up to 15360 × 8640
- Supports 8K60 in all supported device families

  Note: Performance on low power devices can be lower.

IP Facts

AMD LogiCORE™ IP Facts Table	
Core Specifics	
Supported Device Family[1]	AMD Versal™ Adaptive SoC, AMD UltraScale+™ Families, AMD UltraScale™ Families, AMD Zynq™ 7000 SoC, 7 series AMD Versal™ AI Edge Series Gen 2 and AMD Versal™ Prime Series Gen 2 series (tile mode)
Supported User Interfaces	AXI4-Master, AXI4-Lite, AXI4-Stream[2]
Resources	• **Read:** Performance and Resource Utilization for Video Frame Buffer Read web page • **Write:** Performance and Resource Utilization for Video Frame Buffer Write web page
Provided with Core	
Design Files	Not Provided
Example Design	Yes
Test Bench	Not Provided
Constraints File	Xilinx Design Constraints (XDC)
Simulation Model	Encrypted RTL
Supported S/W Driver[3]	Standalone Linux DMA Controller
Tested Design Flows[4]	
Design Entry	AMD Vivado™ Design Suite
Simulation	For supported simulators, see *Vitis Software Platform Release Notes* (UG1742).
Synthesis	VivadoSynthesis
Support	
Release Notes and Known Issues	Master Answer Record • **Read:** 68764 • **Write:** 68765
All Vivado IP Change Logs	Master Vivado IP Change Logs: 72775
Support web page	

Notes:

1. For a complete list of supported devices, see the AMD Vivado™ IP catalog.
2. Video protocol as defined in the Video IP: AXI Feature Adoption section of *Vivado Design Suite: AXI Reference Guide* (UG1037).
3. Linux OS and driver support information is available from the Linux Video Frame Buffer Read Driver Page and Linux Video Frame Buffer Write Driver Page.
4. For the supported versions of third-party tools, see *Vitis Software Platform Release Notes* (UG1742).

Overview

Many video applications require frame buffers to handle frame rate changes or changes to the image dimensions (scaling or cropping). The Video Frame Buffer Read and Video Frame Buffer Write IP cores are designed to allow for efficient high-bandwidth access between the AXI4-Stream video interface and the AXI4 interface.

Navigating Content by Design Process

AMD Adaptive Computing documentation is organized around a set of standard design processes to help you find relevant content for your current development task. You can access the AMD Versal™ adaptive SoC design processes on the Design Hubs page. You can also use the Design Flow Assistant to better understand the design flows and find content that is specific to your intended design needs. This document covers the following design processes:

- **Hardware, IP, and Platform Development:** Creating the PL IP blocks for the hardware platform, creating PL kernels, functional simulation, and evaluating the AMD Vivado™ timing, resource use, and power closure. Also involves developing the hardware platform for system integration. Topics in this document that apply to this design process include:
  - Port Descriptions
  - Register Space
  - Clocking
  - Resets
  - Customizing and Generating the Core
  - Chapter 6: Example Design

Feature Summary

The Video Frame Buffer Read and Video Frame Buffer Write are independent IPs, which support reading and writing a variety of video formats (RGB, YUV 4:4:4, YUV 4:2:2, YUV 4:2:0, and Luma only). The data is packed/unpacked based on the video format. Planar, semi-planar, and 3 planar memory formats are available for YUV 4:2:2, YUV 4:2:0, and YUV 4:4:4. The memory video format, stride, and frame buffer address are runtime programmable.

Applications

Applications range from video systems for broadcast, professional Audio/Video, consumer medical, surveillance, and industrial applications, which include the following functionalities:

- Video switchers, format converters
- Video CODECs
- Video surveillance systems
- Machine vision systems
- Video conferencing end unit
- Set-top box

Licensing and Ordering

This AMD LogiCORE™ IP module is provided at no additional cost with the AMD Vivado™ Design Suite under the terms of the End User License.

For more information about this core, visit the Video Frame Buffer product web page.

Information about other AMD LogiCORE™ IP modules is available at the Intellectual Property page. For information about pricing and availability of other AMD LogiCORE IP modules and tools, contact your local sales representative.

Product Specification

Standards

The Video Frame Buffer Read and Video Frame Buffer Write cores are compliant with the AXI4-Stream Video Protocol, AXI4-Lite interconnect, and memory mapped AXI4 interface standards. For additional information, see the Video IP: AXI Feature Adoption section of the *Vivado Design Suite: AXI Reference Guide* (UG1037).

Performance

The following sections detail the performance characteristics of the Video Frame Buffer Read and Video Frame Buffer Write cores.

Maximum Frequencies

The following are typical clock frequencies for the target devices:

- AMD UltraScale+™ devices with -1 speed grade or higher: 300 MHz
- AMD Virtex™ 7 and AMD Virtex™ UltraScale™ devices with -2 speed grade or higher: 300 MHz
- AMD Kintex™ 7 and Kintex UltraScale devices with -2 speed grade or higher: 300 MHz
- AMD Artix™ 7devices with -2 speed grade or higher: 150 MHz
- AMD Versal™ Adaptive SoC with -1 speed grade or higher: 300 MHz

The maximum achievable clock frequency can vary. The maximum achievable clock frequency and all resource counts can be affected by other tool options, additional logic in the device, using a different version of AMD tools, and other factors.

Resource Use

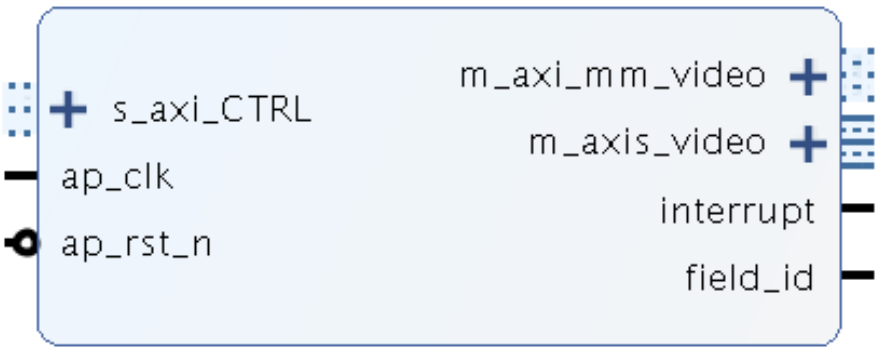
For full details about performance and resource use, visit the Video Frame Buffer Read Performance and Resource Utilization web page and the Video Frame Buffer Write Performance and Resource Utilization web page.

Port Descriptions

The Video Frame Buffer Read and Video Frame Buffer Write cores use industry standard control and data interfaces to connect to other system components. The following sections describe the various interfaces available with the cores. The following figure illustrates the Video Frame Buffer Read diagram. Each IP has three AXI interfaces:

- AXI4-Lite control interface (`s_axi_CTRL`)
- AXI4-Stream streaming video output (`m_axis_video`) or input (`s_axis_video`)
- Memory mapped AXI4 interface (`m_axi_mm_video`)

Figure 1: **Video Frame Buffer Read I/O Diagram**



The following figure illustrates the Video Frame Buffer Write diagrams.

Figure 2: **Video Frame Buffer Write I/O Diagram**

Common Interface Signals

The following table summarizes the signals, which are either shared by, or not part of the dedicated AXI4-Stream, memory mapped AXI4 data, or AXI4-Lite control interfaces.

Table 1: **Common Interface Signals**

Signal Name	I/O	Width	Description
ap_clk	I	1	Video core clock
ap_rst_n	I	1	Video core active-Low clock enable
interrupt	O	1	Interrupt Request Pin
field_id	O (Frame Buffer Read)	1	Field polarity (only available when interlaced support is selected)
	I (Frame Buffer Write)		

The `ap_clk` and `ap_rst_n` signals are shared between the core, the AXI4-Stream, memory mapped AXI4 data interface, and the AXI4-Lite control interface.

ap_clk

The AXI4-Stream, memory mapped AXI4, and AXI4-Lite interfaces must be synchronous to the core clock signal `ap_clk`. All AXI4-Stream, memory mapped AXI4 interface input signals, and AXI4-Lite control interface input signals are sampled on the rising edge of `ap_clk`. All AXI4-Stream output signal changes occur after the rising edge of `ap_clk`.

ap_rst_n

The `ap_rst_n` pin is an active-Low, synchronous reset input pertaining to AXI4-Lite, AXI4-Stream, and memory mapped AXI4 interfaces. When `ap_rst_n` is set to 0, the core resets at the next rising edge of `ap_clk`.

interrupt

The interrupt status output bus can be integrated with an external interrupt controller, which has independent interrupt enable/mask, interrupt clear, and interrupt status registers that allow for interrupt aggregation to the system processor.

field_id

The following table provides information on how the `field_id` signal toggles in the frame buffer read IP.

Table 2: **Fid Output Mode**

FidOutMode	Field ID
0	As per the value mentioned in register Field ID
1	For first field, the field_id signal is low. After that for each field, the field_id toggles
2	For the first two fields, the field_id signal is low. After that for each field the field id toggles

fid error

The fid error register is defined in the frame buffer read IP only. This register should be used only if fid output mode is selected as 1 or 2. This error register is set if the `field_id` signal is not the same as mentioned in the field ID register 0x0048. Otherwise, the value in this register is zero.

AXI4-Stream Video Interface

The Video Frame Buffer Read and Video Frame Buffer Write cores have either an AXI4-Stream video input or output interface named `s_axis_video` or `m_axis_video`, respectively. All video streaming interfaces follow the interface specification as defined in the *AXI4-Stream Video IP and System Design Guide* (UG934). The video AXI4-Stream interface can be single, dual, quad, or octa pixels per clock and can support 8, 10, or 12-bit per component.

The following tables explain the pixel mapping of an AXI4-Stream interface with two pixels per clock and 10 bits per component configuration for all supported color formats. Given that the Video Frame Buffer Read and Video Frame Buffer Write always require a hardware configuration of three component video, the AXI4-Stream Subset Converter is needed to hook up with other IPs of two or one component video interface in YUV 4:2:2, YUV 4:2:0, or Luma-Only.

Table 3: **Dual Pixels per Clock, 10 Bits per Component Mapping for RGB**

63:60	59:50	49:40	39:30	29:20	19:10	9:0
zero padding	R1	B1	G1	R0	B0	G0

Table 4: **Dual Pixels per Clock, 10 Bits per Component Mapping for YUV 4:4:4**

63:60	59:50	49:40	39:30	29:20	19:10	9:0
zero padding	V1	U1	Y1	V0	U0	Y0

Table 5: **Dual Pixels per Clock, 10 Bits per Component Mapping for YUV 4:2:2**

63:60	59:50	49:40	39:30	29:20	19:10	9:0
zero padding	zero padding	zero padding	V0	Y1	U0	Y0

Table 6: **Dual Pixels per Clock, 10 Bits per Component Mapping for YUV 4:2:0, for Even Lines**

63:60	59:50	49:40	39:30	29:20	19:10	9:0
zero padding	zero padding	zero padding	V0	Y1	U0	Y0

Table 7: **Dual Pixels per Clock, 10 Bits per Component Mapping for YUV 4:2:0, for Odd Lines**

63:60	59:50	49:40	39:30	29:20	19:10	9:0
zero padding	zero padding	zero padding	zero padding	Y1	zero padding	Y0

This IP always generates three video components even if the video format is set to be YUV 4:2:0 or YUV 4:2:2 at runtime. The unused components can be set to zero.

The following table shows the interface signals for input and output AXI4-Stream video streaming interfaces.

Table 8: **AXI4-Stream Interface Signals**

Signal Name	I/O	Width	Description
s_axis_tdata	I	floor(((number_of_components × bits_per_component × pixels_per_clock) + 7) / 8) × 8	Input Data
s_axis_tready	O	1	Input Ready
s_axis_tvalid	O	1	Input Valid
s_axis_tdest	I	1	Input Data Routing Identifier

Table 8: **AXI4-Stream Interface Signals** *(cont'd)*

Signal Name	I/O	Width	Description
s_axis_tkeep	I	(s_axis_video_tdata width) / 8	Input byte qualifier that indicates whether the content of the associated byte of TDATA is processed as part of the data stream.
s_axis_tlast	I	1	Input End of Line
s_axis_tstrb	I	(s_axis_video_tdata width) / 8	Input byte qualifier that indicates whether the content of the associated byte of TDATA is processed as a data byte or a position byte.
s_axis_tuser	I	1	Input Start of Frame
m_axis_tdata	O	floor(((number_of_components × bits_per_component × pixels_per_clock) + 7) / 8) × 8	Output Data
m_axis_tdest	O	1	Output Data Routing Identifier
m_axis_tid	O	1	Output Data Stream Identifier
m_axis_tkeep	O	(m_axis_video_tdata width) / 8	Output byte qualifier that indicates whether the content of the associated byte of TDATA is processed as part of the data stream.
m_axis_tlast	O	1	Output End of Line
m_axis_tready	I	1	Output Ready
m_axis_tstrb	O	(m_axis_video_tdata width) / 8	Output byte qualifier that indicates whether the content of the associated byte of TDATA is processed as a data byte or a position byte.
m_axis_tuser	O	1	Output Start of Frame
m_axis_tvalid	O	1	Output Valid

All video streaming interfaces run at the IP core clock speed, that is, `ap_clk`.

Memory Mapped AXI4 Interface

The memory mapped AXI4 interface runs on the `ap_clk` clock domain. The signals follow the specification as defined in the *Vivado Design Suite: AXI Reference Guide* (UG1037). The Video Frame Buffer Read and Video Frame Buffer Write support the pixel formats with raster scan order in memory as described in the following table.

Table 9: **Pixel Formats with Raster Scan Order**

Video Format	ID	Description	Bits per Component	Bytes per Pixel
RGBX8	10	packed RGB	8	4 bytes per pixel
BGRX8	27	packed BGR	8	4 bytes per pixel

Table 9: **Pixel Formats with Raster Scan Order** *(cont'd)*

Video Format	ID	Description	Bits per Component	Bytes per Pixel
YUVX8	11	packed YUV 4:4:4	8	4 bytes per pixel
YUYV8	12	packed YUV 4:2:2	8	2 bytes per pixel
UYVY8	28	packed YUV 4:2:2	8	2 bytes per pixel
RGBA8	13	packed RGB with alpha	8	4 bytes per pixel
BGRA8	26	packed BGR with alpha	8	4 bytes per pixel
YUVA8	14	packed YUV 4:4:4 with alpha	8	4 bytes per pixel
RGBX10	15	packed RGB	10	4 bytes per pixel
YUVX10	16	packed YUV 4:4:4	10	4 bytes per pixel
Y_UV8	18	semi-planar YUV 4:2:2	8	1 byte per pixel per plane
Y_UV8_420	19	semi-planar YUV 4:2:0	8	1 byte per pixel per plane
RGB8	20	packed RGB	8	3 bytes per pixel
BGR8	29	packed BGR	8	3 bytes per pixel
YUV8	21	packed YUV 4:4:4	8	3 bytes per pixel
Y_UV10	22	semi-planar YUV 4:2:2	10	4 bytes per 3 pixels per plane
Y_UV10_420	23	semi-planar YUV 4:2:0	10	4 bytes per 3 pixels per plane
Y8	24	packed luma only	8	1 byte per pixel
XY10	25	packed luma only	10	4 bytes per 3 pixels
RGBX12	30	packed RGB	12	5 bytes per pixel
YUVX12	31	packed YUV 4:4:4	12	5 bytes per pixel
Y_UV12	32	semi-planar YUV 4:2:2	12	3 bytes per 2 pixels
Y_UV12_420	33	semi-planar YUV 4:2:0	12	3 bytes per 2 pixels
Y12	34	packed luma only	12	3 bytes per 2 pixels
RGB16	35	packed RGB	16	6 bytes per pixel
YUV16	36	packed YUV 4:4:4	16	6 bytes per pixel
Y_UV16	37	semi-planar YUV 4:2:2	16	2 bytes per pixel
Y_UV16_420	38	semi-planar YUV 4:2:0	16	2 bytes per pixel
Y16	39	packed luma only	16	2 bytes per pixel
Y_U_V8	42	3 planar YUV 4:4:4	8	1 byte per pixel
Y_U_V8_420	41	3 PLANAR YUV 4:2:0	8	1 byte per pixel
Y_U_V10	43	3 planar YUV 4:4:4	10	4 byte per 3 pixels
Y_U_V12	44	3 planar YUV 4:4:4	12	3 bytes per 2 pixels

The following table shows the IP format in raster order with the corresponding V4L2 and DRM pixel formats.

Table 10: **IP Format to Common Video Pixel Format**

IP Format	V4L2 4CC	DRM 4CC
RGBX8	V4L2_PIX_FMT_BGRX32	DRM_FORMAT_XBGR8888

Table 10: **IP Format to Common Video Pixel Format** *(cont'd)*

IP Format	V4L2 4CC	DRM 4CC
BGRX8	V4L2_PIX_FMT_XRGB32	DRM_FORMAT_XRGB8888
YUVX8	V4L2_PIX_FMT_XVUY32	DRM_FORMAT_XVUY8888
YUYV8	V4L2_PIX_FMT_YUYV	DRM_FORMAT_YUYV
UYVY8	V4L2_PIX_FMT_UYVY	DRM_FORMAT_UYVY
RGBA8	Not supported	Not supported
BGRA8	Not supported	Not supported
YUVA8	Not supported	Not supported
RGBX10	V4L2_PIX_FMT_XBGR30	DRM_FORMAT_XBGR2101010
YUVX10	V4L2_PIX_FMT_XVUY10	DRM_FORMAT_XVUY2101010
Y_UV8	V4L2_PIX_FMT_NV16	DRM_FORMAT_NV16
Y_UV8_420	V4L2_PIX_FMT_NV12	DRM_FORMAT_NV12
RGB8	V4L2_PIX_FMT_RGB24	DRM_FORMAT_BGR888
BGR8	V4L2_PIX_FMT_BGR24	DRM_FORMAT_RGB888
YUV8	V4L2_PIX_FMT_VUY24	DRM_FORMAT_VUY888
Y_UV10	V4L2_PIX_FMT_XV20M	DRM_FORMAT_XV20
Y_UV10_420	V4L2_PIX_FMT_XV15M	DRM_FORMAT_XV15
Y8	V4L2_PIX_FMT_GREY	DRM_FORMAT_Y8
XY10	V4L2_PIX_FMT_XY10[1]	DRM_FORMAT_Y10
RGBX12	V4L2_PIX_FMT_XBGR40[1]	Not supported
YUVX12	Not supported	Not supported
Y_UV12	V4L2_PIX_FMT_X212[1] V4L2_PIX_FMT_X212M[1]	Not supported
Y_UV12_420	V4L2_PIX_FMT_X012[1] V4L2_PIX_FMT_X012M[1]	Not supported
Y12	V4L2_PIX_FMT_XY12[1]	Not supported
RGB16	V4L2_PIX_FMT_BGR48	Not supported
YUV16	Not Supported	Not supported
Y_UV16	V4L2_PIX_FMT_X216[1] V4L2_PIX_FMT_X216M[1]	Not supported
Y_UV16_420	V4L2_PIX_FMT_X016[1] V4L2_PIX_FMT_X016M[1]	Not supported
Y16	V4L2_PIX_FMT_Y16	Not supported

Notes:

1. AMD only formats M – Noncontiguous planes (Chroma plane does not start at the end of Luma plane).

The following tables explain the expected pixel mappings in memory for each of the mentioned listed formats.

***Note*:** RGB formats are stored in memory in RGB order, which is different from the pixel mapping in *AXI4-Stream Video IP and System Design Guide* (UG934).

RGBX8

Packed RGB, 8 bits per component. Every RGB pixel in memory is represented with 32 bits, as shown. The images need to be stored in memory in raster order, that is, the top-left pixel first and the bottom-right pixel last. Bits[31:24] do not contain pixel information.

31:24	23:16	15:8	7:0
X	B	G	R

BGRX8

Packed BGR, 8 bits per component. Every BGR pixel in memory is represented with 32 bits, as shown. The images need to be stored in memory in raster order, that is, the top-left pixel first and the bottom-right pixel last. Bits[31:24] do not contain pixel information.

31:24	23:16	15:8	7:0
X	R	G	B

YUVX8

Packed YUV 4:4:4, 8 bits per component. Every YUV 4:4:4 pixel in memory is represented with 32 bits, as shown. Bits[31:24] do not contain pixel information.

31:24	23:16	15:8	7:0
X	V	U	Y

YUYV8

Packed YUV 4:2:2, 8 bits per component. Every two YUV 4:2:2 pixels in memory are represented with 32 bits, as shown.

31:24	23:16	15:8	7:0
V0	Y1	U0	Y0

UYVY8

Packed YUV 4:2:2, 8 bits per component. Every two YUV 4:2:2 pixels in memory are represented with 32 bits, as shown.

31:24	23:16	15:8	7:0
Y1	V0	Y0	U0

RGBA8

Packed RGB with alpha, eight bits per component. Every RGB with an alpha pixel is represented with 32 bits, as shown. Bits[31:24] contain alpha information, zero is fully transparent, and 255 is fully opaque.

31:24	23:16	15:8	7:0
A	B	G	R

BGRA8

Packed BGR with alpha, eight bits per component. Every BGR with an alpha pixel is represented with 32 bits, as shown. Bits[31:24] contain alpha information, 0 is fully transparent, and 255 is fully opaque.

31:24	23:16	15:8	7:0
A	R	G	B

YUVA8

Packed YUV with alpha, eight bits per component. Every YUV 4:4:4 with an alpha pixel is represented with 32 bits, as shown. Bits[31:24] contain alpha information, 0 is fully transparent, and 255 is fully opaque.

31:24	23:16	15:8	7:0
A	V	U	Y

RGBX10

Packed RGB, 10 bits per component. Every RGB pixel is represented with 32 bits, as shown. Bits[31:30] do not contain any pixel information.

31:30	29:20	19:10	9:0
X	B	G	R

YUVX10

Packed YUV 4:4:4, 10 bits per component. Every YUV 4:4:4 pixel is represented with 32 bits, as shown. Bits[31:30] do not contain any pixel information.

31:30	29:20	19:10	9:0
X	V	U	Y

Y_UV8

Semi-planar YUV 4:2:2 with eight bits per component. Y and UV are stored in separate planes, as shown. The UV plane is assumed to have an offset of stride × height bytes from the Y plane buffer address. See *Stride* in Prerequisites for more information on stride.

31:24	23:16	15:8	7:0
Y3	Y2	Y1	Y0

31:24	23:16	15:8	7:0
V2	U2	V0	U0

Y_UV8_420

Semi-planar YUV 4:2:0 with eight bits per component. Y and UV are stored in separate planes, as shown. The UV plane is assumed to have an offset of stride × height bytes from the Y plane buffer address. See *Stride* in Prerequisites for more information on stride.

31:24	23:16	15:8	7:0
Y3	Y2	Y1	Y0

31:24	23:16	15:8	7:0
V4	U4	V0	U0

Y_U_V8

Three Planar YUV 4:4:4 with eight bits per component. Y, U, and V are stored in 3 separate planes, as shown.

31:24	23:16	15:8	7:0
Y3	Y2	Y1	Y0

31:24	23:16	15:8	7:0
U3	U2	U1	U0

31:24	23:16	15:8	7:0
V3	V2	V1	V0

Y_U_V8_420

Three Planar YUV 4:2:0 with eight bits per component. Y, U, and V are stored in 3 separate planes, as shown in the following table.

31:24	23:16	15:8	7:0

Y3	Y2	Y1	Y0

31:24	23:16	15:8	7:0
U3	U2	U1	U0

31:24	23:16	15:8	7:0
V3	V2	V1	V0

Y_U_V10

Three Planar YUV 4:4:4 with 10 bits per component. Y, U, and V are stored in three separate planes, as shown.

31:30	29:20	19:10	9:0
X	Y2	Y1	Y0

31:30	29:20	19:10	9:0
X	U2	U1	U0

31:30	29:20	19:10	9:0
X	V2	V1	V0

RGB8

Packed RGB, eight bits per component. Every RGB pixel in memory is represented with 24 bits, as shown. The images need to be stored in memory in raster order, that is, the top-left pixel first and the bottom-right pixel last.

23:16	15:8	7:0
B	G	R

BGR8

Packed BGR, eight bits per component. Every RGB pixel in memory is represented with 24 bits, as shown. The images need to be stored in memory in raster order, that is, the top-left pixel first and the bottom-right pixel last.

23:16	15:8	7:0
B	G	R

YUV8

Packed YUV 4:4:4, eight bits per component. Every YUV 4:4:4 pixel in memory is represented with 24 bits, as shown. The images need to be stored in memory in raster order, that is, the top-left pixel first and the bottom-right pixel last.

23:16	15:8	7:0
V	U	Y

Y_UV10

Semi-planar YUV 4:2:2 with 10 bits per component. Every three pixels is represented with 32 bits. Bits[31:30] do not contain any pixel information. Y and UV are stored in separate planes as shown. The UV plane is assumed to have an offset of stride x height bytes from the Y plane buffer address. See *Stride* in Prerequisites for more information on stride.

63:62	61:52	51:42	41:32	31:30	29:20	19:10	9:0
X	Y5	Y4	Y3	X	Y2	Y1	Y0

63:62	61:52	51:42	41:32	31:30	29:20	19:10	9:0
X	V4	U4	V2	X	U2	V0	U0

Y_UV10_420

Semi-planar YUV 4:2:0 with 10 bits per component. Every three pixels is represented with 32 bits. Bits[31:30] do not contain any pixel information. Y and UV are stored in separate planes as shown. The UV plane is assumed to have an offset of stride x height bytes from the Y plane buffer address. See *Stride* in Prerequisites for more information on stride.

63:62	61:52	51:42	41:32	31:30	29:20	19:10	9:0
X	Y5	Y4	Y3	X	Y2	Y1	Y0

63:62	61:52	51:42	41:32	31:30	29:20	19:10	9:0
X	V8	U8	V4	X	U4	V0	U0

Y8

Packed Luma-Only, eight bits per component. Every luma-only pixel in memory is represented with eight bits, as shown. The images need to be stored in memory in raster order, that is, the top-left pixel first and the bottom-right pixel last. Y8 is presented as YUV 4:4:4 on the AXI4-Stream interface.

7:0
Y

XY10

Packed Luma-Only, 10 bits per component. Every three luma-only pixels in memory is represented with 32 bits, as shown. The images need to be stored in memory in raster order, that is, the top-left pixel first and the bottom-right pixel last. Y10 is presented as YUV 4:4:4 on the AXI4-Stream interface.

31:30	29:20	19:10	9:0
X	Y2	Y1	Y0

RGBX12

Packed RGB, 12 bits per component. Every RGB pixel in memory is represented with 40 bits, as shown. The images need to be stored in memory in raster order, that is, the top-left pixel first and the bottom-right pixel last. Bits [39:36] do not contain pixel information.

39:36	35:24	23:12	11:0
X	B	G	R

YUVX12

Packed YUV 4:4:4, 12 bits per component. Every YUV 4:4:4 pixel in memory is represented with 40 bits, as shown. Bits [39:36] do not contain pixel information.

39:36	35:24	23:12	11:0
X	V	U	Y

Y_UV12

Semi-planar YUV 4:2:2 with 12 bits per component. Y and UV are stored in separate planes, as shown.

47:36	35:24	23:12	11:0
Y3	Y2	Y1	Y0

47:36	35:24	23:12	11:0
V2	U2	V0	U0

Y_U_V12

Three Planar YUV 4:4:4 with 12 bits per component. Y, U, and V are stored in three separate planes, as shown.

47:36	35:24	23:12	11:0

Y3	Y2	Y1	Y0

47:36	35:24	23:12	11:0
U3	U2	U1	U0

47:36	35:24	23:12	11:0
V3	V2	V1	V0

Y_UV12_420

Semi-planar YUV 4:2:0 with 12 bits per component. Y and UV are stored in separate planes, as shown.

47:36	35:24	23:12	11:0
Y3	Y2	Y1	Y0

47:36	35:24	23:12	11:0
V4	U4	V0	U0

Y12

Packed Luma-Only, 12 bits per component. Every four luma-only pixels in memory is represented with 48-bit, as shown. The images need to be stored in memory in raster order, that is, the top-left pixel first and bottom-right pixel last. Y12 is presented as YUV 4:4:4 on the AXI4-Stream interface.

47:36	35:24	23:12	11:0
Y3	Y2	Y1	Y0

RGB16

Packed RGB, 16 bits per component. Every RGB pixel in memory is represented with 48 bits, as shown. The images need to be stored in memory in raster order, that is, the top-left pixel first and the bottom-right pixel last.

47:32	31:16	15:0
B	G	R

YUV16

Packed YUV 4:4:4, 16 bits per component. Every YUV 4:4:4 pixel in memory is represented with 48 bits, as shown.

47:32	31:16	15:0

V	U	Y

Y_UV16

Semi-planar YUV 4:2:2 with 16 bits per component. Y and UV are stored in separate planes, as shown.

63:48	47:32	31:16	15:0
Y3	Y2	Y1	Y0

63:48	47:32	31:16	15:0
V2	U2	V0	U0

Y_UV16_420

Semi-planar YUV 4:2:0 with 16 bits per component. Y and UV are stored in separate planes, as shown.

63:48	47:32	31:16	15:0
Y3	Y2	Y1	Y0

63:48	47:32	31:16	15:0
V4	U4	V0	U0

Y16

Packed Luma-Only, 16 bits per component. Every luma-only pixel in memory is represented with 16 bits, as shown. The images need to be stored in memory in raster order, that is, the top-left pixel first and the bottom-right pixel last. Y16 is presented as YUV 4:4:4 on the AXI4-Stream interface.

15:0
Y

***Note*:** There is no timer in the frame buffer to determine the frame's start and end points. It relies solely on interrupts to determine the frame's start and end. Effective management of input frame size is crucial. If a higher frame size is specified but lower values are provided, it can lead to delays as the core waits for additional data. Conversely, declaring a lower frame size while providing more input can result in data loss as the core does not process the excess data.

Tile Format

Tiling is a method of re-arranging the pixels of a surface so that pixels that are close in the two-dimensional euclidean space are more likely to be stored close to each other in the memory. A tiled image is formed by dividing the image pixels into two-dimensional blocks or tiles. Each tile takes up a chunk of contiguous memory. Tiles are stored in raster scan order from left to right and from top to bottom within each plane.

$Tile_0$	$Tile_1$	$Tile_2$	...	$Tile_{N-2}$	$Tile_{N-1}$	
$Tile_N$	$Tile_{N+1}$	$Tile_{N+2}$	...	$Tile_{2N-2}$	$Tile_{2N-1}$	
$Tile_{2N}$	$Tile_{2N+1}$	$Tile_{2N+2}$	...	$Tile_{3N-2}$	$Tile_{3N-1}$	
...	...	...	...	...	...	

***Note*:** Tile Format is supported only in the AMD Versal™ AI Edge Gen 2 and AMD Versal™ Prime Gen 2 series devices where Video Codec Unit (VCU2) AMD LogiCORE™ is available as a Hard IP. For more information, see *Versal AI Edge Gen 2 H.264/H.265/JPEG Video Codec Unit 2 Solutions LogiCORE IP Product Guide* (PG447).

The frame buffer read and write IPs support memory video format in two tile sizes: 32 × 4 and 64 × 4.

- **32 × 4 Tiled Format:**

  With B corresponding to a block of 4 × 4 pixel components, the sample order within a 32 × 4 tile corresponds to a sequence of eight 4 × 4 blocks:

B0	B1	B2	B3	B4	B5	B6	B7

- **64 × 4 Tiled Format:**

  The sample order within a 64 × 4 tile corresponds to a sequence of sixteen 4 × 4 blocks:

B0	B1	B2	B3	B4	B5	B6	B7	B8	B9	B10	B11	B12	B13	B14	B15

  The sample order within each 4 × 4 block B corresponds to the raster scan order.

  - **Luma:**

    The order of pixel components within a 4 × 4 block in luma plane is shown below:

$Y_{x,0}$	$Y_{x,1}$	$Y_{x,2}$	$Y_{x,3}$
$Y_{x,4}$	$Y_{x,5}$	$Y_{x,6}$	$Y_{x,7}$
$Y_{x,8}$	$Y_{x,9}$	$Y_{x,A}$	$Y_{x,B}$
$Y_{x,C}$	$Y_{x,D}$	$Y_{x,E}$	$Y_{x,F}$

  - **Chroma Non-Interleaved:**

With C corresponding to either Cb or Cr block, the order of pixel components within a 4 × 4 block in Cb or Cr plane is shown below:

$C_{x,0}$	$C_{x,1}$	$C_{x,2}$	$C_{x,3}$
$C_{x,4}$	$C_{x,5}$	$C_{x,6}$	$C_{x,7}$
$C_{x,8}$	$C_{x,9}$	$C_{x,A}$	$C_{x,B}$
$C_{x,C}$	$C_{x,D}$	$C_{x,E}$	$C_{x,F}$

- **Chroma Interleaved:**

  The order of pixel components within a 4 × 4 block in interleaved Chroma plane is shown below:

$Cb_{x,0}$	$Cr_{x,0}$	$Cb_{x,1}$	$Cr_{x,1}$
$Cb_{x,2}$	$Cr_{x,2}$	$Cb_{x,3}$	$Cr_{x,3}$
$Cb_{x,4}$	$Cr_{x,4}$	$Cb_{x,5}$	$Cr_{x,5}$
$Cb_{x,6}$	$Cr_{x,6}$	$Cb_{x,7}$	$Cr_{x,7}$

  Consequently, in a 4:2:0 picture, there are half as many chroma tiles as luma tiles because the chroma plane has half the vertical size of the luma plane besides having the same horizontal size, as Cb and Cr components are interleaved. In 4:4:4 and 4:2:2 pictures, there are equal number of chroma and luma tiles. The 4:0:0 monochrome picture consists of only luma tiles.

  The pixel components within each tile are fully packed so the memory footprint of a tile is:

  - 128 bytes for an 8-bit 32 × 4 tile
  - 160 bytes for a 10-bit 32 × 4 tile
  - 192 bytes for a 12-bit 32 × 4 tile
  - 256 bytes for an 8-bit 64 × 4 tile
  - 320 bytes for a 10-bit 64 × 4 tile
  - 384 bytes for a 12-bit 64 × 4 tile

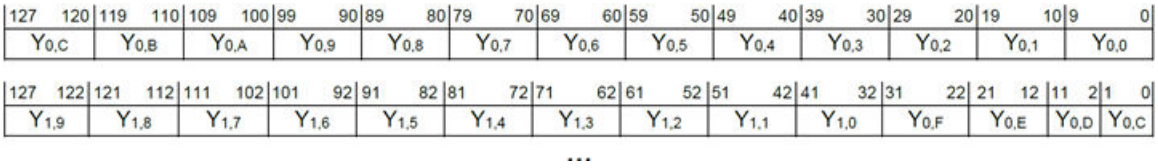
The following tables explain the expected pixel mappings in memory for each of the supported tile formats.

- **8-bit Tile Format:** 8-bit samples are packed starting from the least significant bits of memory words. The footprint of an 8-bit 32x4 tile is 128 bytes (8 x 128-bit words) and the footprint of an 8-bit 64x4 tile is 256 bytes (16 x 128-bit words). An example of tile buffer for luma samples is as follows:

127 120	119 112	111 104	103 96	95 88	87 80	79 72	71 64	63 56	55 48	47 40	39 32	31 24	23 16	15 8	7 0
$Y_{0,F}$	$Y_{0,E}$	$Y_{0,D}$	$Y_{0,C}$	$Y_{0,B}$	$Y_{0,A}$	$Y_{0,9}$	$Y_{0,8}$	$Y_{0,7}$	$Y_{0,6}$	$Y_{0,5}$	$Y_{0,4}$	$Y_{0,3}$	$Y_{0,2}$	$Y_{0,1}$	$Y_{0,0}$
$Y_{1,F}$	$Y_{1,E}$	$Y_{1,D}$	$Y_{1,C}$	$Y_{1,B}$	$Y_{1,A}$	$Y_{1,9}$	$Y_{1,8}$	$Y_{1,7}$	$Y_{1,6}$	$Y_{1,5}$	$Y_{1,4}$	$Y_{1,3}$	$Y_{1,2}$	$Y_{1,1}$	$Y_{1,0}$

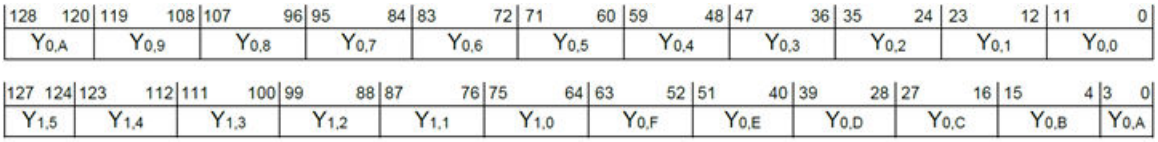
...

- **10-bit Tile Format:** 10-bit samples are packed starting from the least significant bits of memory words. The footprint of a 10-bit 32x4 tile is 160 bytes (10 x 128-bit words) and the footprint of a 10-bit 64x4 tile is 320 bytes (20 x 128-bit words). An example of tile buffer for luma samples is as follows:



...

- **12-bit Tile Format:** 12-bit samples are packed starting from the least significant bits of memory words. The footprint of a 12-bit 32x4 tile is 192 bytes (12 x 128-bit words) and the footprint of a 12-bit 64x4 tile is 384 bytes (24 x 128-bit words). An example of tile buffer for luma samples is as follows:



...

The Video Frame Buffer Read and Video Frame Buffer Write support the pixel formats with tile scan order in memory as described in the following table.

Table 11: **Pixel Formats with Tile Scan Order**

Video Format	Description	Bits per Component	Bytes per 32x4 Tile	Bytes per 64x4 Tile
Y_U_V8	3 planar YUV 4:4:4	8	128	256
Y_U_V10	3 planar YUV 4:4:4	10	160	320
Y_U_V12	3 planar YUV 4:4:4	12	192	384
Y_UV8	semi-planar YUV 4:2:2	8	128	256
Y_UV10	semi-planar YUV 4:2:2	10	160	320
Y_UV12	semi-planar YUV 4:2:2	12	192	384
Y_UV8_420	semi-planar YUV 4:2:0	8	128	256
Y_UV10_420	semi-planar YUV 4:2:0	10	160	320
Y_UV12_420	semi-planar YUV 4:2:0	12	192	384
Y8	monochrome YUV 4:0:0	8	128	256
Y10	monochrome YUV 4:0:0	10	160	320
Y12	monochrome YUV 4:0:0	12	192	384

***Note*:** IP Format to Common Video Pixel Format is not supported for tile mode.

AXI4-Lite Control Interface

The AXI4-Lite interface allows you to control parameters within the core dynamically. The configuration can be accomplished using an AXI4-Lite master state machine, an embedded Arm®, or a soft system processor such as MicroBlaze™. The Video Frame Buffer Read and Video Frame Buffer Write cores can be controlled through the AXI4-Lite interface by using functions provided by the driver in the MicroBlaze™ software platform. Another method is performing read and write transactions to the register space. It should only be used when the first method is not available. The following table shows the AXI4-Lite control interface signals. This interface runs at the `ap_clk` clock.

Table 12: **AXI4-Lite Control Interface Signals**

Signal Name	I/O	Width	Description
s_axi_ctrl_aresetn	I	1	Reset
s_axi_ctrl_aclk	I	1	Clock
s_axi_ctrl_awaddr	I	18	Write Address
s_axi_ctrl_awprot	I	3	Write Address Protection
s_axi_ctrl_awvalid	I	1	Write Address Valid
s_axi_ctrl_awready	O	1	Write Address Ready
s_axi_ctrl_wdata	I	32	Write Data
s_axi_ctrl_wstrb	I	4	Write Data Strobe
s_axi_ctrl_wvalid	I	1	Write Data Valid
s_axi_ctrl_wready	O	1	Write Data Ready
s_axi_ctrl_bresp	O	2	Write Response
s_axi_ctrl_bvalid	O	1	Write Response Valid
s_axi_ctrl_bready	I	1	Write Response Ready
s_axi_ctrl_araddr	I	18	Read Address
s_axi_ctrl_arprot	I	3	Read Address Protection
s_axi_ctrl_arvalid	I	1	Read Address Valid
s_axi_ctrl_aready	O	1	Read Address Ready
s_axi_ctrl_rdata	O	32	Read Data
s_axi_ctrl_rresp	O	2	Read Data Response
s_axi_ctrl_rvalid	O	1	Read Data Valid
s_axi_ctrl_rready	I	1	Read Data Ready

Register Space

The Video Frame Buffer Read and Video Frame Buffer Write cores have specific registers that allow you to control the operation of the cores dynamically. All registers have an initial value of zero.

Top-Level Registers

The following table provides a detailed description of all the registers that apply globally to the IP.

Table 13: **Top-Level Registers**

Address (hex) BASEADDR+	Register Name	Access Type	Register Description
0x0000	Control	R/W	Bit [0] = ap_start (R/W/COH)[1] Bit [1] = ap_done (R/COR) Bit [2] = ap_idle (R) Bit [3] = ap_ready (R) Bit [5] = Flush pending AXI transactions (R/W) Bit [6] = Flush done (R) Bit [7] = auto_restart (R/W) Others = reserved
0x0004	Global Interrupt Enable	R/W	Bit [0] = Global interrupt enable Others = Reserved
0x0008	IP Interrupt Enable	R/W	Bit [0] = ap_done Bit [1] = ap_ready Others: reserved
0x000c	IP Interrupt Status	R/TOW [1]	Bit [0] = ap_done Bit [1] = ap_ready Others: reserved
0x0010	Width	R/W	Active width of stream on s_axis_video or m_axis_video
0x0018	Height	R/W	Active height of stream on s_axis_video or m_axis_video
0x0020	Stride	R/W	Active stride (in bytes)
0x0028	Memory Video Format	R/W	Active video format of data in memory
0x0030	Plane 1 Buffer	R/W	Start address of plane 1 of frame buffer
0x003C	Plane 2 Buffer	R/W	Start address of plane 2 of frame buffer. Only valid when a semi-planar video format is selected.
0x0048	Field ID	R/W	Field polarity. Only valid for interlaced video.

Table 13: **Top-Level Registers** *(cont'd)*

Address (hex) BASEADDR+	Register Name	Access Type	Register Description
0x0050	Fid Output Mode	R/W	Applicable for Frame buffer read IP only. Defines the `field_id` toggle mode. Bit [1:0]: 0 = Default mode. The `field_id` signal is based on the field ID value defined in register 0x0048. Bit [1:0]: 1 = For first field, the `field_id` signal is low. After that for each field, the `field_id` toggles Bit [1:0]: 2 = The first two fields, the `field_id` signal is low. After that for each field the field id toggles.
0x0054	Frame Buffer Write Plane 3 Buffer	R/W	Start address of plane 3 of frame buffer. Only valid when a 3 planar video format is selected.
0x0058	fid error	R	Applicable for frame buffer read IP only. Generates error for the discrepancy of the Field ID register and `field_id` signal. Bit[0]: Error bit
0x0074	Frame Buffer Read Plane 3 Buffer	R/W	Start address of plane 3 of frame buffer. Only valid when a 3 planar video format is selected.

Notes:

1. COH = Clear on Handshake, COR = Clear on Read, TOW = Toggle on Write.
2. Status Register (0x000C) are explained in section S_AXILITE Control Register Map of *Vitis High-Level Synthesis User Guide* (UG1399). These registers definitions may have some additional bits; however, in the current IP, we are accessing only bits mentioned in the table. Therefore, only these bits need to be considered while accessing the Control Register, Global Interrupt Enable Register, IP Interrupt Enable Register, and IP Interrupt Status Register.

Control (0x0000) Register

This register controls the operation of the core.

- Bit[0] of the Control register, `ap_start`, kicks off the core from software. Writing 1 to this bit starts the core to generate a video frame.
- Bit[1] of the Control register, `ap_done`, indicates when the IP has completed all operations in the current transaction. A logic 1 on this signal indicates that the IP has completed all operations in this transaction.
- Bit[2] of the Control register, `ap_idle`, signal indicates if the IP is operating or idle (no operation). The idle state is indicated by logic 1. This signal is asserted low once the IP starts operating. This signal is asserted high when the IP completes the operation and no further operations are performed.

- Bit[3] of the Control register, `ap_ready`, signal indicates when the IP is ready for new inputs. It is set to logic 1 when the IP is ready to accept new inputs, indicating that all input reads for this transaction are completed. If the IP has no operations in the pipeline, new reads are not performed until the next transaction starts. This signal is used to make a decision on when to apply new values to the input ports and whether to start a new transaction.
- Bit[5] of the Control register, is for flushing pending AXI transactions. This bit should be set and held (until reset) by software to flush pending transactions. When this is set, the hardware is expecting a hard reset.
- Bit[6] of the Control register is the flush status bit and is asserted when the flush is done.
- Bit[7] of the Control register, `auto_restart`, can be set to enable the auto-restart mode, and then the IP restarts automatically at the end of each transaction.

Global Interrupt Enable (0x0004) Register

This register is the master control for all interrupts. Bit[0] can be used to enable/disable all core interrupts.

IP Interrupt Enable (0x0008) Register

This register allows interrupts to be enabled selectively. Currently, two interrupt sources are available, `ap_done` and `ap_ready`. `ap_done` is triggered after the frame processing is complete, while `ap_ready` is triggered after the core is ready to start processing the next frame.

IP Interrupt Status (0x000C) Register

This is a dual-purpose register. When an interrupt occurs, the corresponding interrupt source bit is set in this register. In readback mode (`Get status`), the interrupting source can be determined. In writeback mode (`Clear interrupt`), the requested interrupt source bit is cleared.

Width (0x0010) Register

The `Width` register encodes the number of Active Pixels per Scanline. Supported values are between 64 and the value provided in the *Maximum Number of Columns* field in the AMD Vivado™ Integrated Design Environment (IDE). To avoid processing errors, you should restrict values written to `Width` to the range supported by the core instance. Furthermore, `Width` needs to be a multiple of the *Samples per Clock* field in the Vivado IDE for the raster scan order and multiple of both four and the *Samples per Clock* field for the tiled order.

Height (0x0018) Register

The `Height` register encodes the number of active scan lines per frame (or per field for interlaced video). Supported values are between 64 and the value provided in the *Maximum Number of Rows* field in the Vivado IDE. To avoid processing errors, you should restrict values written to `Height` to the range supported by the core instance. For interlaced data, the height for each field must be the same. Furthermore, `Height` must be a multiple of eight in case of tile scan order.

Stride (0x0020) Register

The stride determines the number of bytes between rows of pixels or rows of tiles in memory based on the following memory video formats.

- **Memory in Raster Order:**

  The stride value in raster order defines the address offset between two consecutive rows of pixels.

- **Memory in Tile Order:**

  The stride value in tile order defines the address offset between two consecutive rows of tiles. A row of tiles corresponds to four rows of pixels. The stride in tile order is also referred to as Pitch.

When a video frame is stored in memory, the memory buffer can contain extra padding bytes after each row of pixels.

Padding bytes are necessary to ensure that every row of pixels starts at an address that aligns with the size of the data on the memory mapped AXI4 interface. In tile format, padding bytes are also necessary when the width of the video frame is not exactly divisible by the width of the tile. The padding bytes only affect how the image is stored in memory and not how it displays.

The stride value needs to be calculated based on order of pixel components in memory. The stride value must be a multiple of the memory-mapped AXI4 data size. The data size of the memory mapped AXI4 interface is 64*Samples per Clock bits, for example, 64, 128, 256, or 512 bits for one, two, four, and eight samples per clock, respectively.

Memory Video Format (0x0028) Register

This register specifies the video format that the core uses for memory. Supported memory video formats are those selected in the Vivado IDE (See Memory Mapped AXI4 Interface). The memory video format should match the AXI4-Stream interface data format as shown in the following table. Memory formats using eight bits per component can be scaled up and then read out to a 10-bit per component video stream. (Scaling is performed by shifting bits. For example, 255 in eight bits becomes 1020 in 10 bits.) Similarly, a 10-bit video stream can be scaled down and written to 8-bit memory video formats. The reverse is not true. 8-bit video stream data cannot be written to or read from 10-bit memory video formats.

Table 14: **Compatible Memory and Streaming Video Formats**

Memory Video Format	Compatible AXI4-Stream Video Format
RGBX8	RGB
RGBX10	
RGB8	
BGR8	
BGRX8	
YUVX8	YUV 4:4:4
YUVX10	
YUV8	
Y_U_V8	
Y_U_V10	
Y8	
Y10	
Y12	
Y_U_V12	
YUYV8	YUV 4:2:2
Y_UV8	
Y_UV10	
UYVY8	
Y_UV12	
Y_UV8_420	YUV 4:2:0
Y_U_V8_420	
Y_UV10_420	
Y_UV12_420	
RGBA8	RGB with per pixel alpha
BGRA8	
YUVA8	YUV 4:4:4 with per pixel alpha

Plane1 Buffer (0x0030), Plane 2 Buffer (0x003C), and Plane3 Buffer (0x0054)

The Plane 1 Buffer register specifies the frame buffer address of plane 1. Note that for the semi-planar formats (Y_UV8, Y_UV8_20, Y_UV10, Y_UV10_420), the chroma buffer is specified by the Plane 2 Buffer register, and for 3 planar formats (Y_U_V8 , Y_U_V10, and Y_U_V8_420), the chroma buffers are specified by the Plane 2 and Plane 3 Buffer registers. The address needs to be aligned with the data size of the memory mapped AXI4 interface. The data size of the memory mapped AXI4 interface is 64*Samples per Clock bits, for example- 64, 128, 256, or 512 bits for one, two, four, and eight samples per clock, respectively.

If the IP is configured to a 64-bit address width in the IP catalog, the IP internally creates another register in the register space by adding 0x4 to the existing buffer address register offset. For example, Plane1 Buffer register addresses are 0x0030 and 0x0034.

Fid Output Mode

The Fid Output Mode register is defined in the frame buffer read IP only. This register defines the mode of `field_id` toggle mode. The following table provides the applicable values for this register to be programmed.

Table 15: **Fid Output Mode**

FidOutMode	Field ID
0	As per the value mentioned in the register Field ID.
1	For the first field, the `field_id` signal is low. After that, for each field, the `field_id` toggles.
2	For the first two fields, the `field_id` signal is low. After that, for each field, the field id toggles.

fid error

The fid error register is defined in the frame buffer read IP only. This register should be used only if the fid output mode is selected as 1 or 2. This error register is set if the `field_id` signal is not the same as mentioned in the field ID register 0x0048. Otherwise, the value in this register will be zero.

Field ID (0x0048) Register

Field ID is used to identify if the field being read/written is even or odd. This register is only available with interlaced video. The register is R/W for the Frame Buffer Read and Read only for the Frame Buffer Write.

Perform the following to access the 64-bit DDR memory location:

1. Change the IP address width to 64-bit in the IP GUI.

2. In Vivado address editor, unmap the HP0_DDR_LOW base name, which has a 0x0000_0000 offset address with a 2G band.
3. Auto-assign addresses to map DDR_LOW and DDR_HIGH address spaces for 64-bit mode.
4. Vivado will get DDR_HIGH offset address as 0x0000_0008_0000_0000 with 32G band. The IP can use any address as a source/destination buffer address.

Designing with the Core

This section includes guidelines and additional information to facilitate designing with the core.

General Design Guidelines

This section compares the differences between Video Frame Buffer Read and Video Frame Buffer Write IPs to the AXI Video Direct Memory Access IP core.

The AXI Video Direct Memory Access (VDMA) does not perform pixel packing or unpacking of data it reads and writes to and from memory. Data is written to memory as it is formatted on the `AXIS_VIDEO_TDATA` bus. This has the following consequences:

- 8-bit RGB is written to memory as GBR.
- YUV 4:2:2 over a three-component streaming interface is written to memory as three components although the third component does not contain any valid data.
- YUV 4:2:0 is the same as YUV 4:2:2. This means 4:2:0 takes twice the bandwidth compared to separating out luma and chroma planes.
- 10-bit formats are written differently to memory depending on pixels per clock. For example, with one pixel per clock, RGBX is written as:

31:30	29:20	19:10	9:0
X	R	B	G

Whereas with two pixels per clock, RGBX is written as:

63:60	59:50	49:40	39:30	29:20	19:10	9:0
X	R	B	G	R	B	G

Video Frame Buffer Read and Video Frame Buffer Write IPs have some differences/limitations when compared to AXI VDMA. The following list highlights some key differences between Video Frame Buffer IP and AXI VDMA.

- Video Frame Buffer Read and Video Frame Buffer Write cores support single channel read or single channel write, but never combined. The AXI VDMA supports combined read and write channels in one IP.

- Because of the previous point, Video Frame Buffer Read and Video Frame Buffer Writecores do not use Genlock Synchronization (prevents reading and writing from and to one buffer). You should implement this in the software.
- Video Frame Buffer Read and Video Frame Buffer Write cores currently only support 8, 10, and 12 bits per component.
- Video Frame Buffer Read and Video Frame Buffer Write cores do not allow for data realignment to the byte (8 bits) level. For example, the addresses must be aligned to multiples of memory mapped data width bytes.
- Video Frame Buffer Read and Video Frame Buffer Write cores do not allow for explicit control of the size of the memory bus. If data width conversion is required, use an AXI interconnect IP.

***Note*:** AMD recommends using the Video Frame Buffer Read and Video Frame Buffer Write for reading from or writing to memory when using Video IP such as the Video Processing Subsystem. For AMD Zynq™ UltraScale+™ MPSoC devices, the Video Codec Unit can properly encode and decode the video data in memory.

Clock, Enable, and Reset Considerations

Clocking

The Video Frame Buffer Read and Video Frame Buffer Write IPs have a single clock domain. All interfaces (master and slave AXI4-Stream video interfaces, the AXI4-Lite interface, and the memory mapped AXI4 interfaces) use the `ap_clk` pin as their clock source.

Resets

The Video Frame Buffer Read and Video Frame Buffer Write IPs have only a hardware reset option, `ap_rst_n` pin. No software reset option is available. The external reset pulse must be held for 16 or more `ap_clk` cycles to reset the core. The `ap_rst_n` signal is synchronous to the `ap_clk` clock domain. The `ap_rst_n` signal resets the entire core, including the AXI4-Lite, AXI4-Stream, and memory mapped AXI4 interfaces.

System Considerations

The Video Frame Buffer read and write IPs must be configured for the actual input and output image frame size to operate properly. The IP can be connected to the Video In to AXI4-Stream input and the Video Timing Controller core to gather the frame size information from the image video stream. The timing detector logic in the Video Timing Controller gathers the video timing signals. The AXI4-Lite control interface on the Video Timing Controller allows the system processor to read out the measured frame dimensions and program all downstream cores, such as the Video Frame Buffer Read and Video Frame Buffer Write, with the appropriate image dimensions.

Ensure that there is sufficient bandwidth available to the Video Frame Buffer Read and Video Frame Buffer Write. The bandwidth required (in MB/s) can be calculated with the following equation:

$$\text{Bandwidth (MB/s)} = \text{fps} \times \text{height} \times \text{stride}$$

where *fps* is the number of frames per second the Video Frame Buffer Read and Video Frame Buffer Writeis operating at; `height` is the height in lines; `stride` is the stride value in bytes. See `Stride` in Prerequisites for more information on stride.

Programming Sequence

The `Buffer` parameters of the IP can be changed dynamically and the change is picked up immediately. If the `Width`, `Height`, `Stride`, or `Video Format` must be changed or the entire system restarted, it is recommended that pipelined AMD IP video cores are disabled/reset from system output towards the system input, and programmed/enabled from system output to system input.

Pending AXI transactions can be flushed before resetting the IP. Assert and hold Bit[5] of the Control register. Bit[6] asserts when the flush is complete and then the IP can be reset.

For interlaced video, with the `ap_ready` interrupt, the current Field ID can be read for the Video Frame Buffer Write IP, and the next Field ID can be programmed for the Frame Buffer Read IP.

Core Behavior with EOL and SOF Signals

As discussed in Chapter 3: Product Specification, the AXI4-Stream video streaming interface for frame buffer read and write cores includes two flags, `tlast` and `tuser`, which are used to identify specific pixels in the video stream.

The `tlast` signal, also known as the end of line (EOL), marks the last valid pixel of each line. This EOL pulse is one valid transaction wide and must align with the last pixel of a scanline. The `tuser` signal, referred to as the start of frame (SOF), designates the first valid pixel of a frame. The SOF pulse is also one valid transaction wide and must coincide with the first pixel of the frame, serving as a frame synchronization signal. These EOL and SOF flags are essential for identifying pixel locations on the AXI4-Stream interface, as there are no sync or blank signals. The frame buffer write core maintains internal registers to track the pixels received per line and the lines received per video frame. The behavior of frame buffer write in case of early or late EOL and SOF flags is explained below, assuming `tvalid` is high when these events occur.

- **Early EOL:** When the EOL signal is asserted prematurely, the core disregards the EOL and continues to receive data until the pixel count per line reaches the frame width specified in the configuration register at offset 0x0010. After receiving the next EOL pulse, the core will reset the pixel count register one cycle later.
- **Late EOL:** If the EOL signal is delayed, the core will ignore any data received after the pixel count per line reaches the frame width set in the configuration register, until the EOL signal is asserted.
- **Early SOF:** When the SOF signal is asserted early, the core initiates the internal line count and pixel count per line, and begins to receive any data arriving on the data bus.
- **Late SOF:** If the SOF signal is asserted late, the core will ignore any data received on the data bus once the internal line count reaches the frame height specified in the configuration register at offset 0x0018, until the SOF signal is asserted.

Design Flow Steps

This section describes customizing and generating the core, constraining the core, and the simulation, synthesis, and implementation steps that are specific to this IP core. More detailed information about the standard AMD Vivado™ design flows and the IP integrator can be found in the following Vivado Design Suite user guides:

- *Vivado Design Suite User Guide: Designing IP Subsystems using IP Integrator* (UG994)
- *Vivado Design Suite User Guide: Designing with IP* (UG896)
- *Vivado Design Suite User Guide: Getting Started* (UG910)
- *Vivado Design Suite User Guide: Logic Simulation* (UG900)

Customizing and Generating the Core

This section includes information about using AMD tools to customize and generate the core in the AMD Vivado™ Design Suite.

If you are customizing and generating the core in the Vivado IP integrator, see the *Vivado Design Suite User Guide: Designing IP Subsystems using IP Integrator* (UG994) for detailed information. IP integrator might auto-compute certain configuration values when validating or generating the design. To check whether the values do change, see the description of the parameter in this chapter. To view the parameter value, run the `validate_bd_design` command in the Tcl console.

You can customize the IP for use in your design by specifying values for the various parameters associated with the IP core using the following steps:

1. Select the IP from the IP catalog.
2. Double-click the selected IP or select the Customize IP command from the toolbar or right-click menu.

For details, see the *Vivado Design Suite User Guide: Designing with IP* (UG896) and the *Vivado Design Suite User Guide: Getting Started* (UG910).

Figures in this chapter are illustrations of the Vivado IDE. The layout depicted here might vary from the current version.

Interface

This section provides a quick reference to parameters that you can configure using the Vivado Design Suite at generation time.

The following figures show the Video Frame Buffer Read and Video Frame Buffer Write Vivado IDE main configuration screens, respectively.

Figure 3: **Video Frame Buffer Read Customize IP**

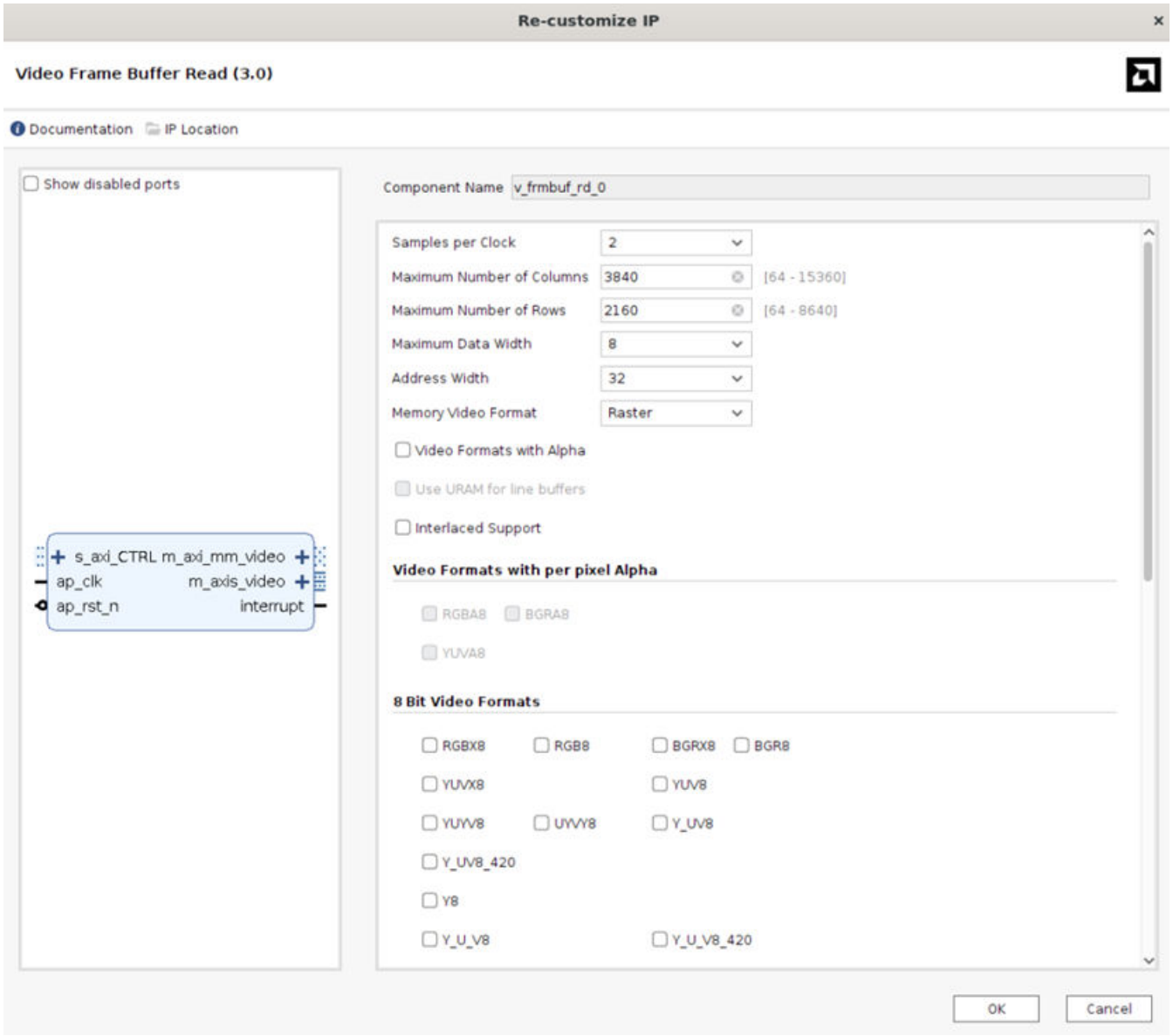
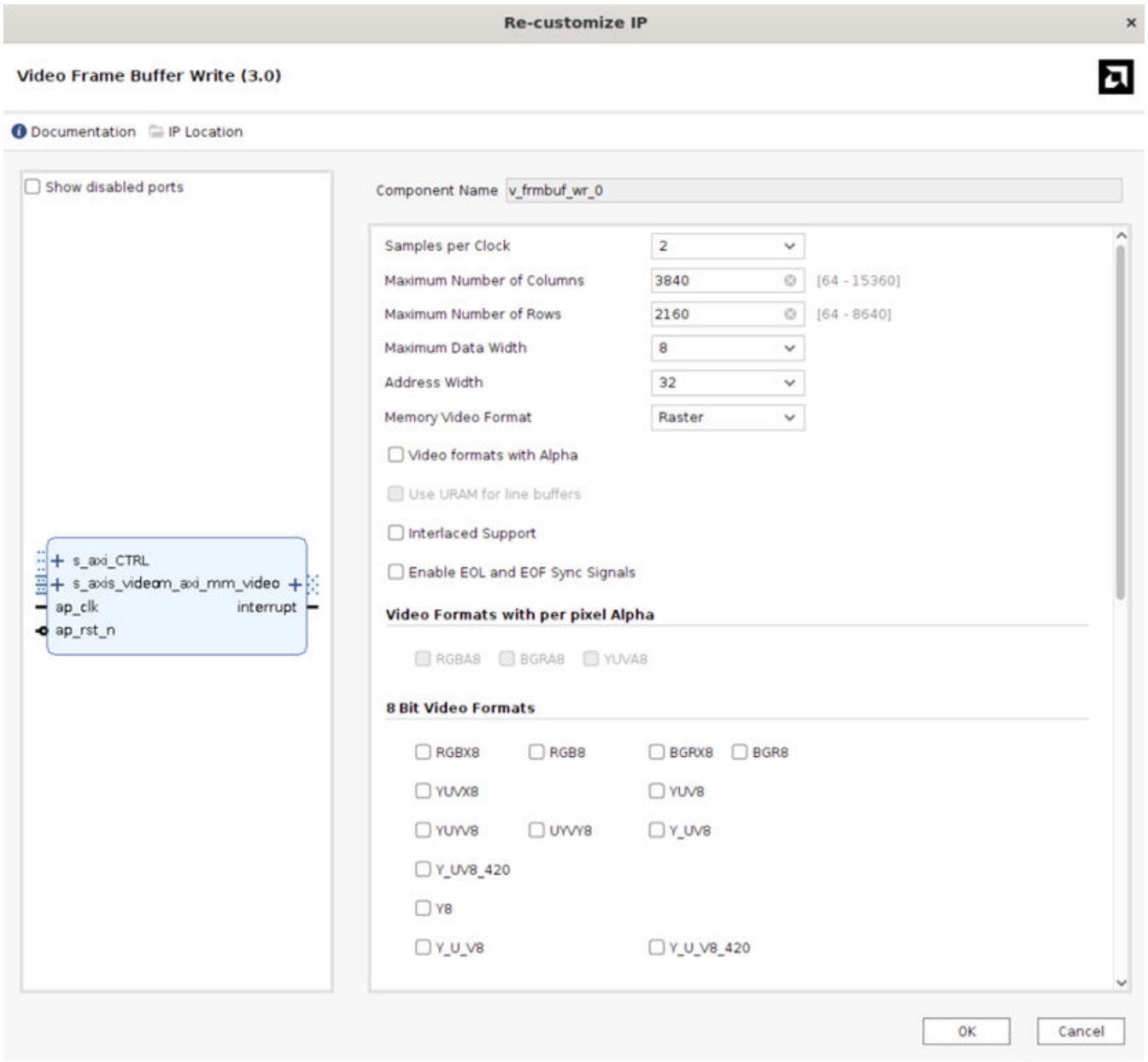


Figure 4: **Video Frame Buffer Write Customize IP**



General Settings

The following settings are generally applicable:

- **Component Name:** The component name is used as the base name of output files generated for the module. Names must begin with a letter and must be composed of characters: a to z, 0 to 9, and "_".

- **Samples Per Clock:** Specifies the number of pixels processed per clock cycle. Permitted values are one, two, four, and eight samples per clock. This parameter determines the IP throughput. The more samples per clock, the larger throughput it provides. The larger throughput always needs more hardware resources.
- **Maximum Number of Columns:** Specifies maximum active video columns/pixels the IP core could produce at runtime. Any video width that is less than the *Maximum Number of Columns* can be programmed through the AXI4-Lite control interface without regenerating the core.
- **Maximum Number of Rows:** Specifies maximum active video rows/lines the IP core could produce at runtime. Any video height that is less than *Maximum Number of Rows* can be programmed through the AXI4-Lite control interface without regenerating the core.
- **Maximum Data Width:** Specifies the bit width of input and output samples on all the streaming interfaces. Permitted values are 8, 10, and 12 bits. This parameter should match the Video Component Width of the video IP core connected to the AXI4-Stream video interface.
- **Memory Video Format:** Specifies the order in which the pixel component data is written into or read from memory. Available formats are raster order and tile order.

  ***Note*:** This option is available only for AMD Versal™ AI Edge Series Gen 2 devices. For all other devices, memory video format is fixed to raster order by default.
- **Address Width:** Specifies the address width of the AXI master interfaces for the memory interface, either 32 or 64 bits.
- **Interlaced Support:** Select for support of interlaced video (in addition to progressive video).
- **Use URAM for Line buffers:** Select this check box to force the tool to use URAM for implementation of line buffers in Tile format. This option is available only for AMD Versal™ AI Edge Series Gen 2 devices when Samples per clock = 8. For other devices or Sample per clock < 8, the tool uses either block RAM or URAM based on the resource availability.
- **Enable EoL and EoF Sync Signals:** This is an optional parameter available only for AMD Versal™ AI Edge Series Gen 2 devices. Enabling this parameter enables two output ports named `sync_eol` and `sync_eof`. Low latency encoding in AMD LogiCORE™ H.264/H.265/ JPEG Video Codec Unit 2 (VCU2) requires the signals from these two ports. The `sync_eol` is a one clock cycle pulse at every end of 16-pixel lines while `sync_eof` is a one clock cycle pulse at every end of frame.
- **Video Formats:**

  Both packed and semi-planar formats are offered. If only packed formats are selected, the IP is smaller in size because there is only one plane. The following formats are packed and only need one plane:

  - RGBX8
  - BGRX8
  - YUVX8

- YUYV8
- UYVY8
- RGBA8
- BGRA8
- YUVA8
- RGBX10
- YUVX10
- RGB8
- BGR8
- YUV8
- Y8
- Y10
- Y12

The following formats are semi-planar and require two planes:

- Y_UV8
- Y_UV8_420
- Y_UV10
- Y_UV10_410
- Y_UV12
- Y_UV12_410

The following formats are 3 planar and require three planes:

- Y_U_V8
- Y_U_V8_420
- Y_U_V10
- Y_U_V12

***Note*:** Tile mode memory format uses line buffers to store the pixel data on chip. The PL RAM usage of line buffers is proportional to the maximum data width, sample per clock, maximum height and maximum width settings. To reduce the PL RAM usage, it is suggested to set these parameters as per your requirements.

Select the memory video formats to be supported. The video format can be programmed through the AXI4-Lite control interface. Details on each video format can be found in the Memory Mapped AXI4 Interface.

User Parameters

The following table shows the relationship between the fields in the Vivado IDE and the User Parameters (which can be viewed in the Tcl Console).

Table 16: **Vivado IDE Parameter to User Parameter Relationship**

Vivado IDE Parameter	User Parameter	Default Value
Number of Video Components	NUM_VIDEO_COMPONENTS	3
Samples per Clock	SAMPLES_PER_CLOCK	2
Maximum Number of Columns	MAX_COLS	3840
Maximum Number of Rows	MAX_ROWS	2160
Maximum Data Width	MAX_DATA_WIDTH	8
AXIMM Data Width	AXIMM_DATA_WIDTH	128
AXIMM Address Width	AXIMM_ADDR_WIDTH	32
Number Read/Write Outstanding	AXIMM_NUM_OUTSTANDING	4
Transaction Burst Length	AXIMM_BURST_LENGTH	16
Video Formats with per pixel Alpha	HAS_ALPHA	0
Memory Video Format	IS_TILE_FORMAT	Raster (0)
Interlaced Support	HAS_INTERLACED	0
Use URAM for line buffer	USE_URAM	0
Enable EoL and EoF Sync Signals	ENABLE_SYNC_SIGNALS	0
RGBX8	HAS_RGBX8	0
BGRX8	HAS_BGRX8	0
YUVX8	HAS_YUVX8	0
YUYV8	HAS_YUYV8	0
UYVY8	HAS_UYVY8	0
RGBA8	HAS_RGBA8	0
BGRA8	HAS_BGRA8	0
YUVA8	HAS_YUVA8	0
RGBX10	HAS_RGBX10	0
YUVX10	HAS_YUVX10	0
Y_UV8	HAS_Y_UV8	0
Y_UV8_420	HAS_Y_UV8_420	0
RGB8	HAS_RGB8	1
BGR8	HAS_BGR8	0
YUV8	HAS_YUV8	0
Y_UV10	HAS_Y_UV10	0
Y_UV10_420	HAS_Y_UV10_420	0
Y_UV12	HAS_Y_UV12	0
Y_UV12_420	HAS_Y_UV12_420	0
Y8	HAS_Y8	0

Table 16: **Vivado IDE Parameter to User Parameter Relationship** *(cont'd)*

Vivado IDE Parameter	User Parameter	Default Value
Y10	HAS_Y10	0
Y12	HAS_Y12	0
Maximum Number of Pixel Planes	MAX_NR_PLANES	1
Y_U_V8	HAS_Y_U_V8	0
Y_U_V8_420	HAS_Y_U_V8_420	0
Y_U_V10	HAS_Y_U_V10	0
Y_U_V12	HAS_Y_U_V12	0

Output Generation

For details, see the *Vivado Design Suite User Guide: Designing with IP* (UG896).

Constraining the Core

Required Constraints

This section is not applicable for this IP core.

Device, Package, and Speed Grade Selections

This section is not applicable for this IP core.

Clock Frequencies

This section is not applicable for this IP core.

Clock Management

This section is not applicable for this IP core.

Clock Placement

This section is not applicable for this IP core.

Banking

This section is not applicable for this IP core.

Transceiver Placement

This section is not applicable for this IP core.

I/O Standard and Placement

This section is not applicable for this IP core.

Simulation

For comprehensive information about AMD Vivado™ simulation components, as well as information about using supported third-party tools, see the *Vivado Design Suite User Guide: Logic Simulation* (UG900).

IMPORTANT! *For cores targeting 7 series or AMD Zynq™ 7000 devices, UNIFAST libraries are not supported. AMD IP is tested and qualified with UNISIM libraries only.*

Synthesis and Implementation

For details about synthesis and implementation, see the *Vivado Design Suite User Guide: Designing with IP* (UG896).

Example Design

This chapter provides two example systems: one containing the Video Frame Buffer Read and the other one containing both the Video Frame Buffer Read and the Video Frame Buffer Write cores. Important system-level aspects when designing with the cores are highlighted in the example designs, including:

- Typical usage of the Video Frame Buffer Read and Video Frame Buffer Write in conjunction with other cores.
- Runtime configuration of the Video Frame Buffer Read and Video Frame Buffer Write by programming registers on-the-fly.

The supported platforms are listed in the following table.

Table 17: **Supported Platforms**

Development Boards	Additional Hardware	Processor
KC705	N/A	MicroBlaze™
ZCU102	N/A	psu_cortexa53_0
ZCU104	N/A	psu_cortexa53_0
ZCU106	N/A	psu_cortexa53_0
VCK190	N/A	CIPS

To open the example project, perform the following:

1. Select the **Video Frame Buffer Read IP** or **Video Frame Buffer Write IP** from the AMD Vivado™ IP catalog.
2. Double-click the selected IP or right-click the IP and select **Customize IP** from the menu.
3. Configure the parameters in the Customize IP window as needed and click **OK**.
4. In the Generate Output Products window, select **Generate** or **Skip**. If Generate is selected, the IP output products are generated after a brief moment.
5. Right-click **Video Frame Buffer Read** or **Video Frame Buffer Write** in the Sources panel and select **Open IP Example Design** from the menu.
6. In the Open IP Example Design window, select the example project directory and click **OK**. The Vivado software then runs automation to generate the example design in the selected directory.

Frame Buffer Read Example Design

The example design delivered with the Frame Buffer Read IP contains the following video IP cores: Video Frame Buffer Read, Video Timing Controller, and AXI4-Stream to Video Output bridge. The design also contains a processor (to enable register programming) connected to an AXI Interconnect. A Memory Interface Generator (MIG) is used for DDR memory access.

The processor acts as the system's AXI master and drives the Video Frame Buffer Read and Video Timing Controller cores. It configures the height, width, and other registers of the Video Frame Buffer Read. It then configures the timing parameters of the Video Timing Controller core.

The Video Frame Buffer Read core generates video stream pixels at a clock rate of `ap_clk`. The core reads frames from a memory mapped AXI4 memory interface and sends the data out over the AXI4-Stream master interface.

The AXI4-Stream to Video Out core, working with the Video Timing Controller, interfaces with the AXI4-Stream interface implementing a timed Video Protocol to a video source (parallel video data with video syncs and blanks).

The example design checks the LOCKED output from the AXI4-Stream to the Video Out core. A locked port indicates that the output timing is locked to the output video. The example design indicates that the test is completed successfully if the video lock is successfully detected. The locked port of AXI4-Stream to Video Out is connected to the AXI GPIO core and the processor polls the corresponding register for a sign that the test passed.

Frame Buffer Write Example Design

The Video Frame Buffer Write core has a separate example design. The difference between the example design delivered with Video Frame Buffer Write and the example design delivered with Video Frame Buffer Read is that the Video Frame Buffer Write design also contains the Video Frame Buffer Write and Video Input to AXI4-Stream bridge. The example design checks the overflow port from the Video Input to AXI4-Stream for data being dropped and not consumed in a timely fashion by the Video Frame Buffer Write core. The overflow port of the Video Input to the AXI4-Stream core is connected to the AXI GPIO for the processor to monitor.

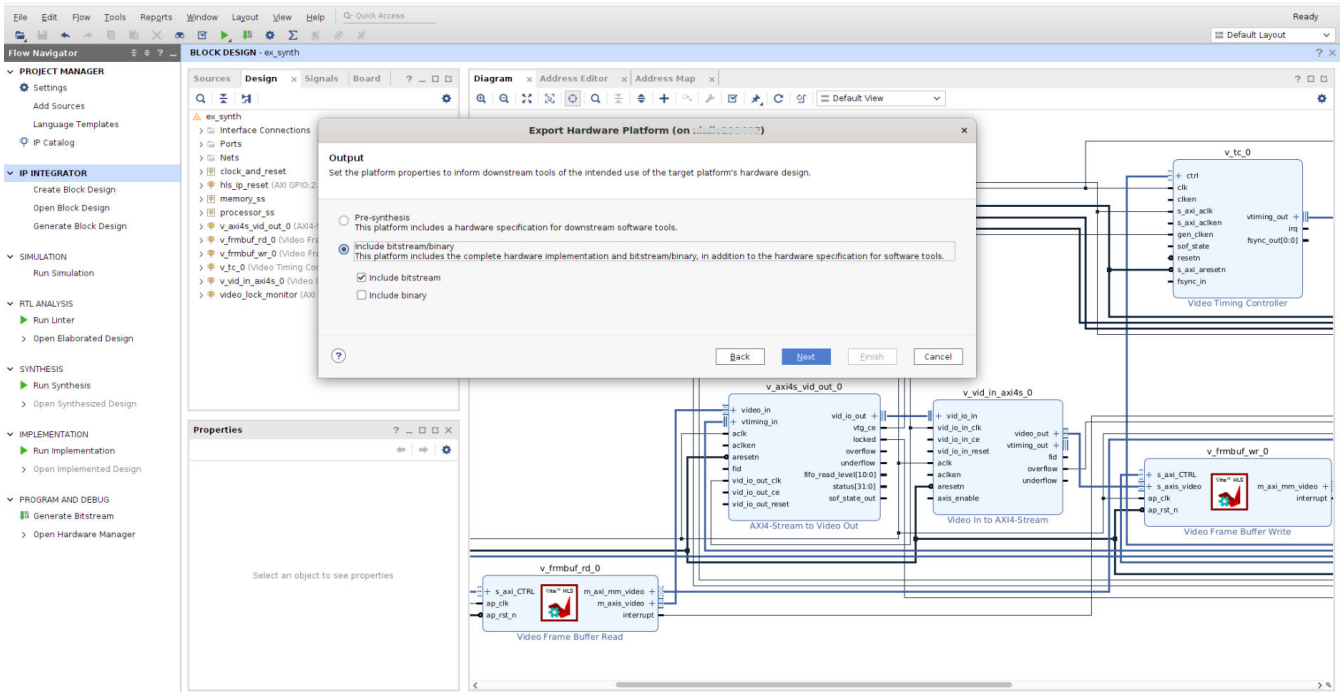
Running the Example Design

The synthesizable example design requires both Vivado and AMD Vitis™ tools. To run the example design, perform the following:

1. Run synthesis, implementation and bitstream generation in AMD Vivado™.

2. After completing step 1, select **File → Export → Export Hardware**. In the window, select **Include bitstream**, select an export directory, and click **OK** to Create XSA.

Figure 5: **Vitis XSA Creation**

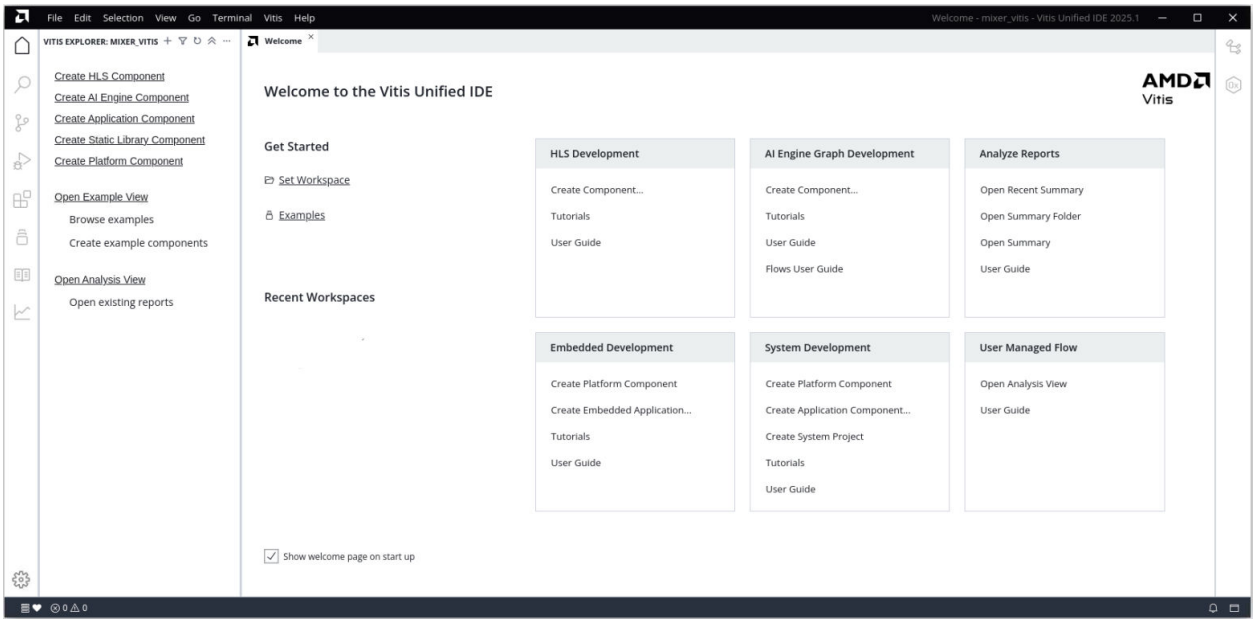


The remaining work is performed in AMD Vitis tool. The example design files can be found in the following Vitis directory:

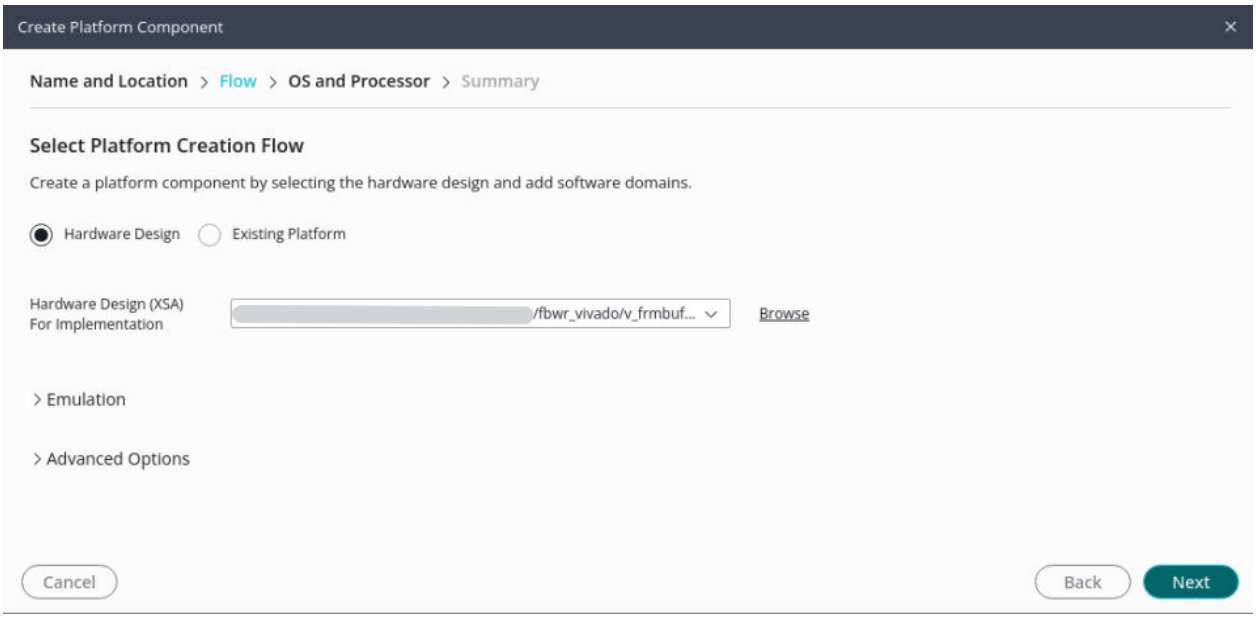
```
(<install_directory>/<release>/data/embeddedsw/
XilinxProcessorIPLib/drivers/
v_frmbuf_rd_v5_0/examples/(<install_directory>/<release>/data/
embeddedsw/XilinxProcessorIPLib/drivers/ v_frmbuf_wr_v5_0/examples/
```

Vitis Software Platform Workflow

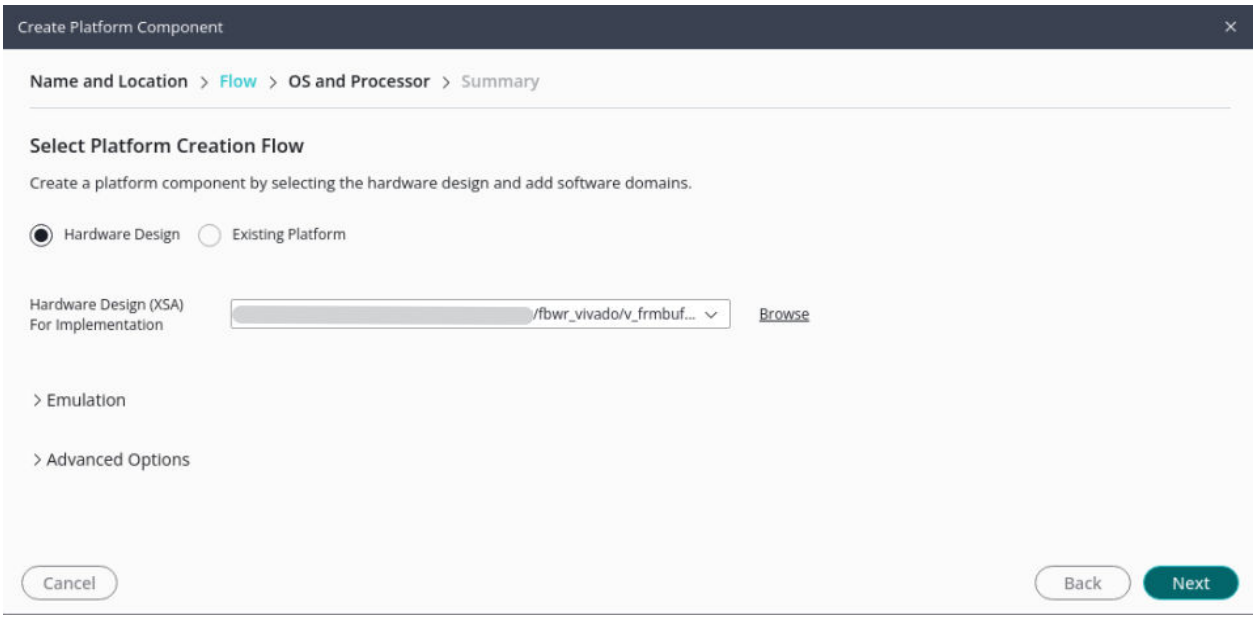
1. Open the Vitis application. Set workspace path by selecting **Get Started → Set Workspace**.



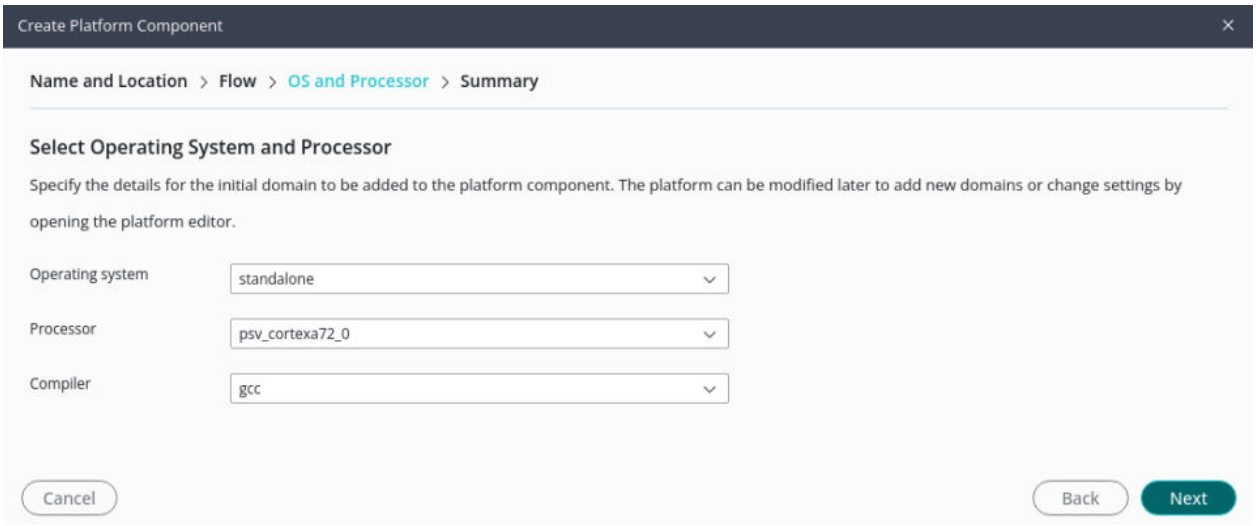
2. Create the platform component by selecting **Embedded Development → Create Platform Component**.
3. Enter the **Component name** and **Component location** in the pop-up window and click **Next**.



4. Select the required XSA.

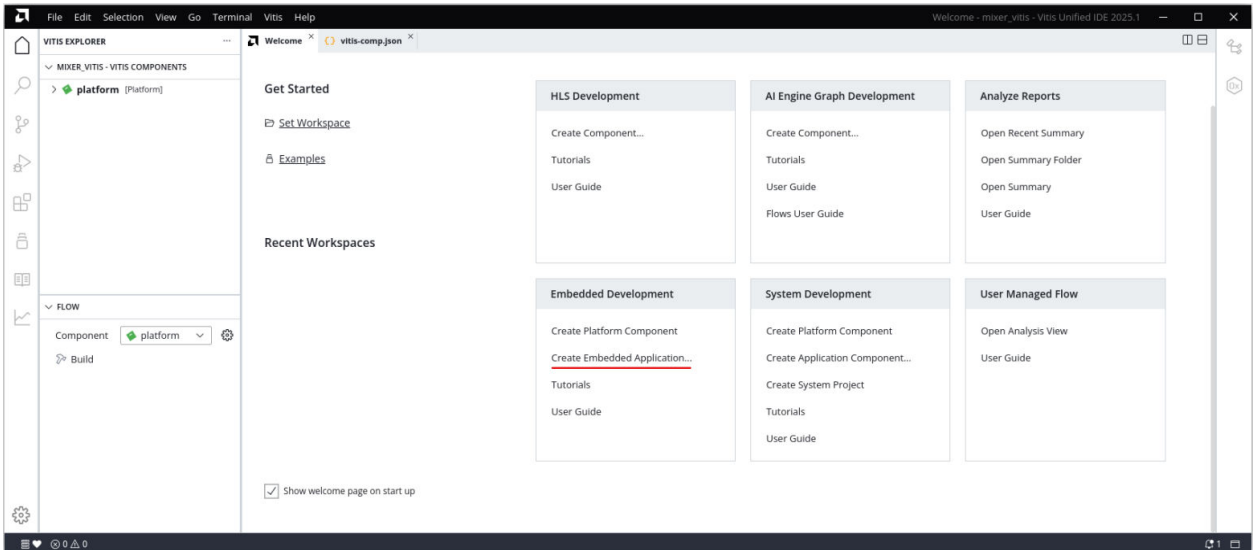


5. Select the **Operating System**, **Processor**, and **Compiler**.

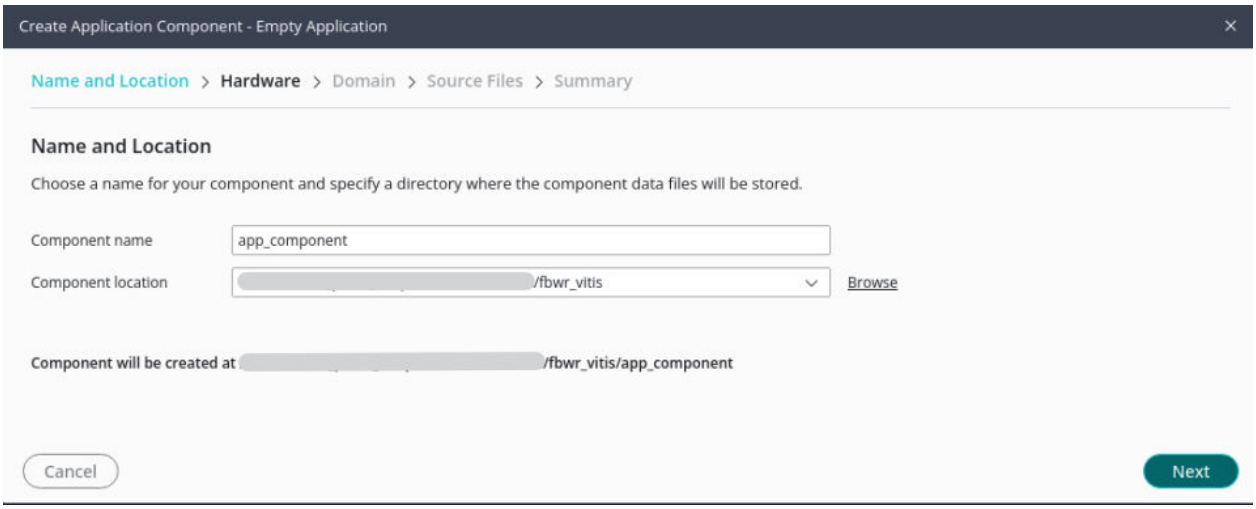


6. Review the summary and click **Finish**.

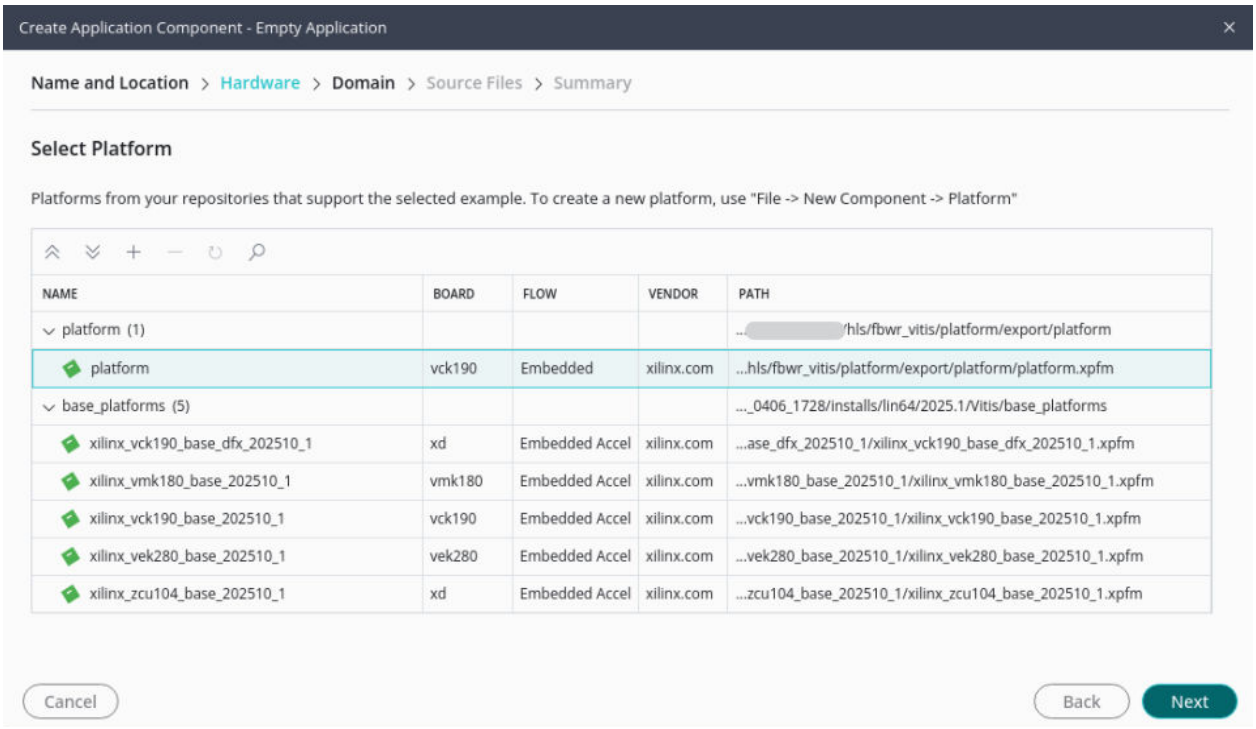
7. Once the platform component is created, switch to the welcome panel, and select the **Create Empty Embedded Application** option in **Embedded Development → Create Platform Component**.



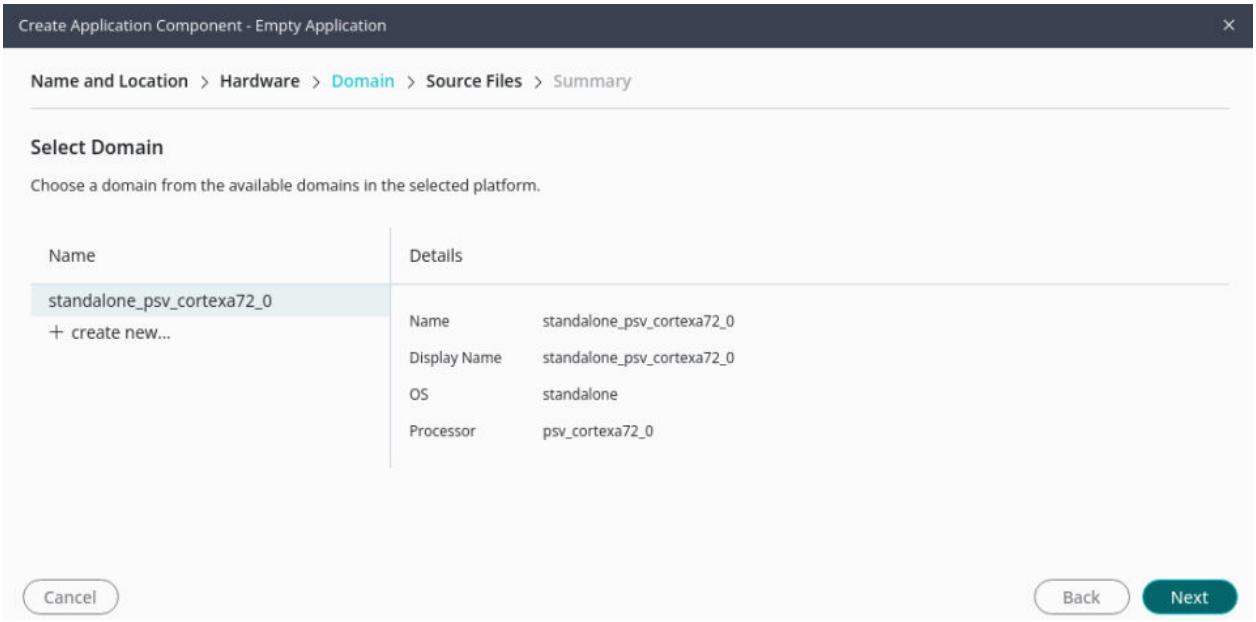
8. Choose the **Component name** and **Component location** in the pop-up window. Select **Next**.



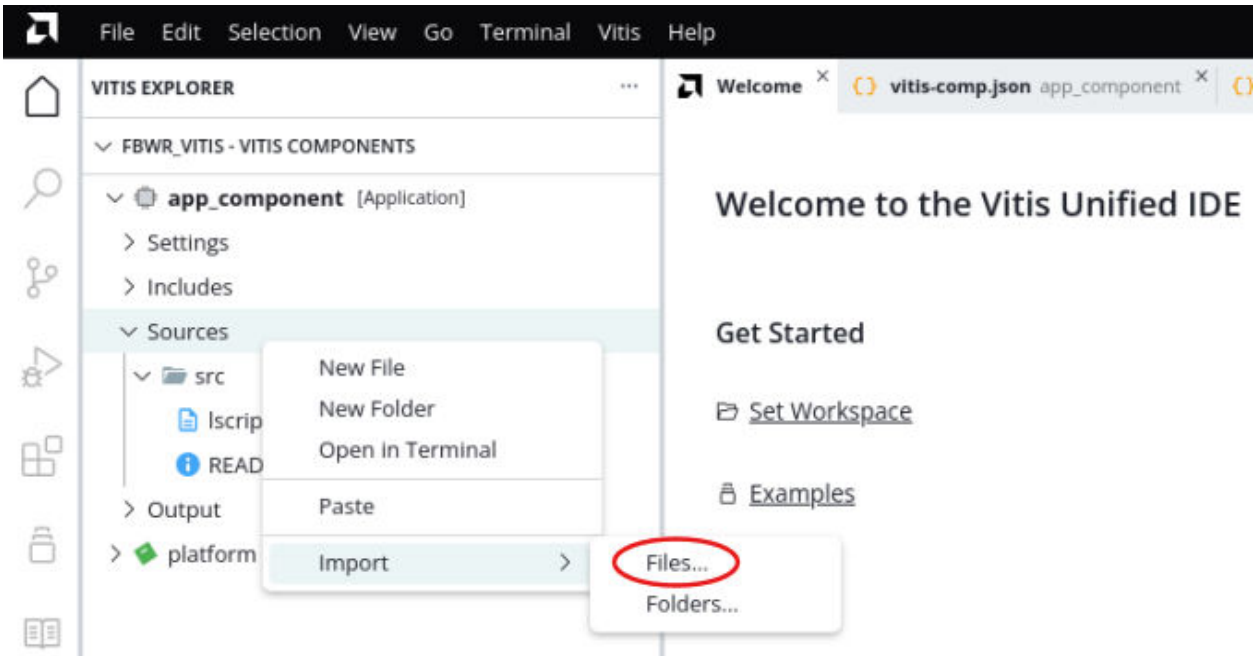
9. Select the required Platform in the hardware tab and click **Next**.



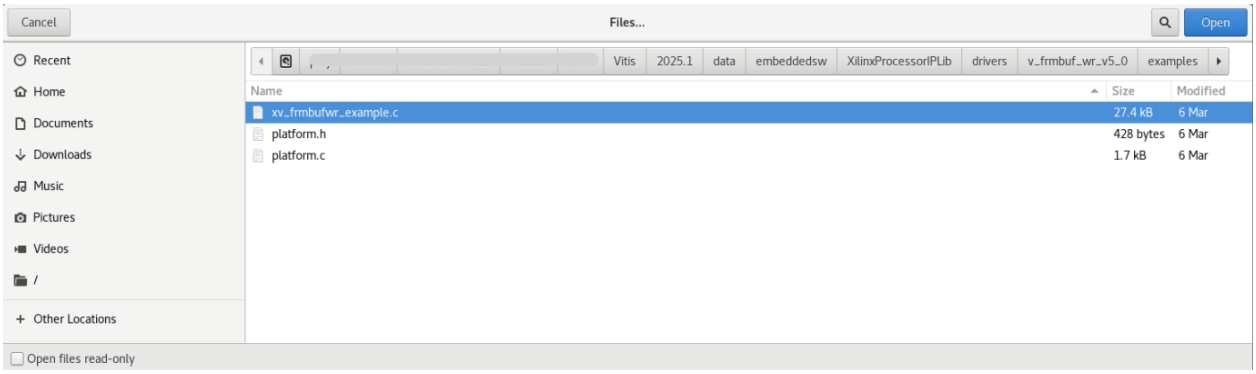
10. Choose the domain from the available domains.



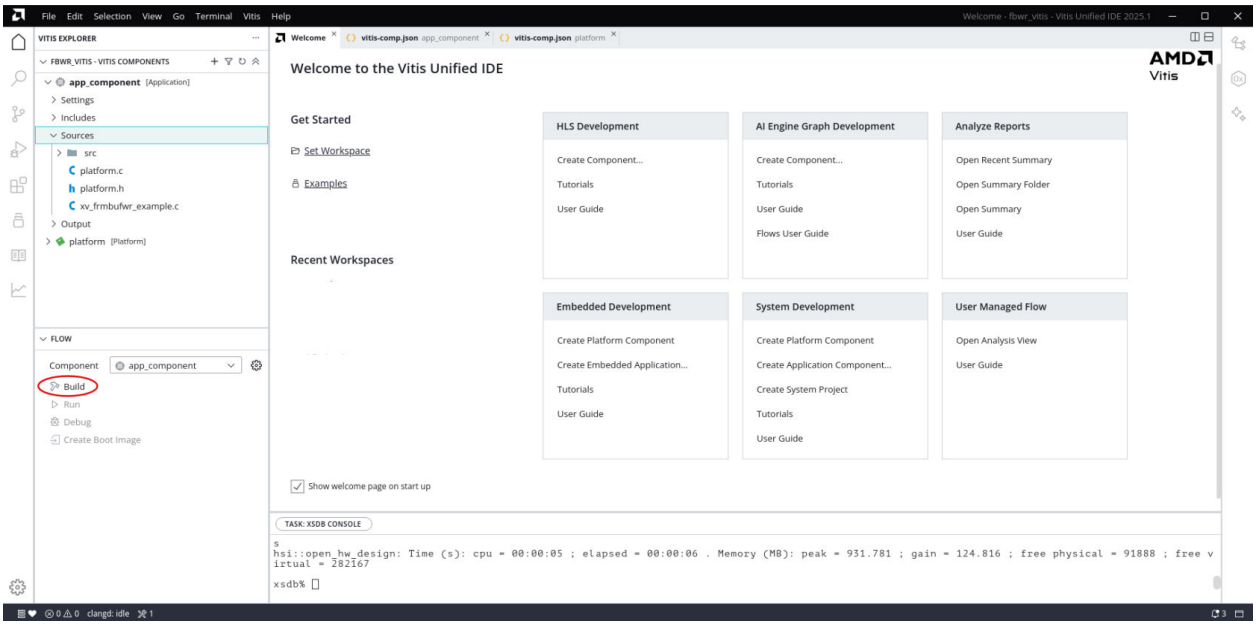
11. Add source files or skip for now, review the summary, and click **Finish**.
12. If skipped in the previous step, import the source files into the app component by right clicking on sources and selecting **Import → Files**.



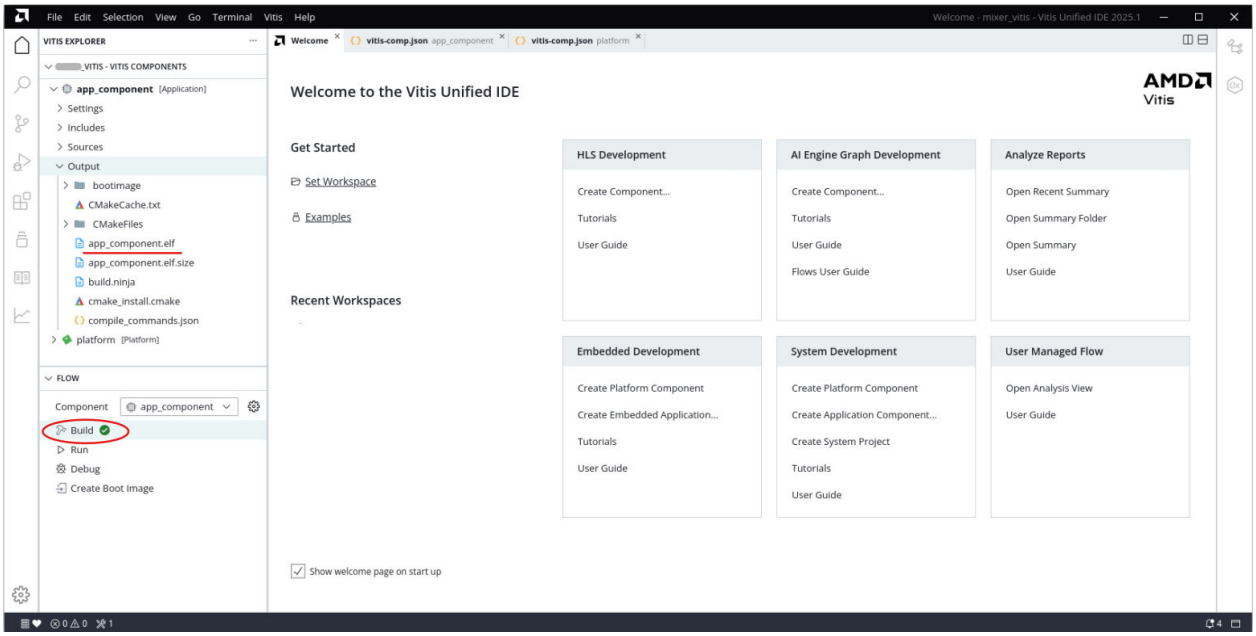
13. Import the required files.



14. Build the project by selecting **Flow → Build**. When prompted for Platform build, select **Yes**.



15. Once the build is complete, check the Output folder for the elf file.



The example application design source files (contained within the examples folder) are tightly coupled with the example design available in Vivado Catalog.

The `v_frmbuf_rd_example.tcl` and `v_frmbuf_wr_example.tcl` files automate the process of generating the downloadable bit and elf files from the provided example HDF file.

To run the provided Tcl script:

1. Copy the exported example design xsa file in the `examples` directory of the driver.

2. Launch the AMD Software Command-Line Tool (xsct) terminal.
3. cd into the examples directory.
4. Source the tcl file in xsct: `%>source vfrmbuf_xx_example.tcl`, where xx is rd for Video Frame Buffer Read and wr for Video Frame Buffer Write.
5. Execute the script: `xsct%>vfrmbuf_xx_example <xsa_file_name.xsa>`

The Tcl script performs the following:

- Create workspace
- Create hardware project
- Create Board Support Package
- Create Application Project
- Build BSP and Application Project

After the process is complete, the required files are available in:

```
bit file -> v_frmbuf_xx_exa/ folder.

elf file -> v_frmbuf_xx_ex/sdk/xv_frmbuf_xx_example_1/{Debug/Release}
folder.
```

Next, perform the following steps to run the software application:

IMPORTANT! *To do so, ensure that the hardware is powered on and a Digilent Cable or a USB Platform Cable is connected to the host PC. Also, ensure that a USB cable is connected to the UART port of the KC705 board.*

1. Launch the Vitis software platform.
2. Set workspace to `vfrmbuf_xx_example.sdk` folder in prompted window. The SDK project opens automatically. (If a welcome page displays, close it.)
3. Download the bitstream into the FPGA by selecting **Xilinx Tools → Program FPGA**. The Program FPGA dialog box opens.
4. Ensure that the Bitstream field shows the bitstream file generated by the Tcl script, then click **Program**.

   ***Note*:** The DONE LED on the board turns green if the programming is successful.
5. A terminal program (such as Hyper Terminal or Putty) is needed for UART communication. Open the program, choose the appropriate port, set the baud rate to 115200, and establish a Serial port connection.
6. Select and right-click the application `vfrmbuf_xx_example_design` in the Project_Explorer panel.
7. Select **Run As → Launch on Hardware (GDB)**.

8. Select **Binaries and Qualifier** in the window and click **OK**.The example design test results are shown in the terminal program.

When executed on the board, the operations are listed in the `readme.txt` in the examples folder. Video inputs tested are 4Kp60, 4Kp30, 1080p, and 720p.

Test Bench

There is no test bench for this IP core release.

Appendix A

Verification, Compliance, and Interoperability

This appendix provides details about how this IP core was tested for compliance with the protocol for which it was designed.

Simulation

A highly parameterizable test bench was used to test the Video Frame Buffer Read and Video Frame Buffer Write in AMD Vitis™ High-Level Synthesis (HLS). Testing included the following:

- Register accesses
- Processing multiple frames of data
- Varying IP throughput and pixel data width
- Testing the Video Frame Buffer Read and Video Frame Buffer Write with AXI4-Stream and memory mapped AXI4 interface
- Testing of various frame sizes
- Varying parameter settings

Hardware Testing

The Video Frame Buffer Read and Video Frame Buffer Write cores have been validated at AMD to represent many different parameterizations. A test design was developed for the core that incorporated a processor, AXI4-Lite interconnect, and various other peripherals. The processor was responsible for:

- Programing the video clock to match the tested video resolution
- Configuring the video IP cores with different resolutions
- Launching the test
- Reporting the Pass/Fail status of the test and any errors that were found

Interoperability

The core slave (input) and master (output) AXI4-Stream interface can work directly with any core that produces RGB, YUV 4:4:4, YUV 4:2:2, or YUV 4:2:0 video data.

Appendix B

Upgrading

This appendix contains information about upgrading to a more recent version of the IP core.

Upgrading in the Vivado Design Suite

This section provides information about any changes to the user logic or port designations between core versions.

Changes from v2.5 to 3.0

Video Frame Buffer v3.0 is a direct replacement for v2.5. V3.0 supports a new video formats: Alpha formats in frame buffer write, Y_U_V12, and Memory Tile format.

Changes from v2.4 to v2.5

Video Frame Buffer v2.5 is a direct replacement for v2.4. V2.5 supports two new video formats: Y_U_V8_420 and Y_U_V8, both 3 Planar video formats.

Changes from v2.3 to v2.4

Video Frame Buffer v2.4 is a direct replacement for v2.3. V2.4 supports a new video format Y_U_V10, that is, a three Planar video format.

Changes from v2.2 to v2.3

Video Frame Buffer v2.3 is a direct replacement for v2.2. V2.3 supports two new features, that is, 16K resolution and Y_U_V8, that is, a three Planar video format.

Changes from v2.1 to v2.2

Video Frame Buffer v2.2 is a direct replacement for v2.1. The IP features remain the same, but AMD Vitis™ HLS tool synthesizes the IP instead of AMD Vivado™ HLS tool.

Changes from v2.0 (Rev. 1) (10/04/2017) to v2.0 (Rev. 2)

Parameters Changes

Two new parameters were added:

- **HAS_BGR8:** The memory format BGR8 has been added.
- **HAS_INTERLACED:** Support for interlaced data in addition to progressive data has been added.

Port Changes

The port `field_id` is added when support for interlaced video is selected.

Other Changes

v2.0 (Rev. 2)

- In v2.0 (Rev. 2) (04/04/2018), there are APIs for both `ap_done` interrupts and `ap_ready` interrupts. The API function calls for the interrupt enable and disable have changed. An additional argument is needed to define a mask for which interrupt to enable/disable. The `SetCallback` function also has an additional argument to specify the interrupt source.
- In v2.0 (Rev. 2) (04/04/2018), the AXI Memory Mapped interface always operates in conservative mode. The HLS AXI master interface always checks the data buffer on the W channel before it issues a write request. Only when the buffer has more than burst length data is `AWVALID` asserted. This can help solve deadlock in some systems but usually, the write latency will increase.
- In v2.0 (Rev. 2) (04/04/2018), pending AXI transactions can be flushed before reset.

v2.0 (Rev. 1)

- In v2.0 (Rev. 1) (10/04/2017), only the `ap_done` interrupt was supported in the driver APIs.
- In v2.0 (Rev. 1) (10/04/2017) does not use conservative mode:

  The HLS AXI master interface issued a write request on the AW channel without checking the data buffer on the W channel.

Changes from v1.0 to v2.0

Parameter Changes

- **New parameters:** HAS_ALPHA, HAS_RGBA8, HAS_BGRA8, HAS_YUVA8. The Video Frame Buffer Read IP supports per pixel Alpha. Two new streaming video formats, RGB with Alpha and YUV 4:4:4 with Alpha, are now available for use with the memory formats RGBA8, BGRA8, and YUVA8.
- **New parameters:** HAS_BGRX8, HAS_UYVY8. New memory video formats have been added to both the Read and Write IPs: BGRX8 and UYVY8.
- **New parameter:** AXIMM_ADDR_WIDTH. Both 32-bit and 64-bit addressing are supported for the memory interfaces.

Port Changes

There are no port changes.

Other Changes

Two buffers are provided for semi-planar formats. In v1.0 of the IP, the chroma buffer address was automatically set based on frame size and stride. In v2.0 of the IP, the chroma buffer address must be specified through a register.

Appendix C

Application Software Development

The Video Frame Buffer Read and Video Frame Buffer Write cores are delivered with a bare-metal driver as part of the AMD Vitis™ installation. The driver follows a layered architecture wherein layer 1 provides basic register peek/poke capabilities and requires users to be familiar with the register map and inner workings of the core. Layer 2, on the other hand, abstracts away all the lower-level details and provides an easy-to-use functional interface to the Video Frame Buffer Read and Video Frame Buffer Write cores. AMD recommends always using layer 2 APIs to interact with the core.

Building the Board Support Package

When the Board Support Package is built, the Video Frame Buffer Read and Video Frame Buffer Write drivers inherently pull in the required dependency, that is, the video common driver. This driver contains enumerations for video-specific information such as color format, color depth, frame rate, and so on. It also defines fundamental data types to represent the video stream and timing that are used in the Video Frame Buffer Read and Video Frame Buffer Write drivers and is common across all AMD video drivers.

During the build process, the Video Frame Buffer Read and Video Frame Buffer Write drivers extract the Video Frame Buffer Read and Video Frame Buffer Write hardware configuration settings from the provided hardware design file.

Prerequisites

There are certain requirements that must be fulfilled when programming the core.

- The core does not contain a data realignment engine; therefore, the application software must align the memory address before writing to the register. The alignment requirement is specified below. This ensures the start address is aligned with the width of the memory interface.

  Note: The start address should be aligned with: (8 * Pixels per Clock) Bytes.

- The Stride value (in bytes) must be aligned as above to make sure that every row of pixels starts at an aligned memory location. Use the following equation to compute the stride from the width (in pixels) in raster-scan order:

  $$\text{Stride in Bytes} \geq (\text{Width} \times \text{Bytes per Pixel});$$

  The bytes per pixel value varies per video in memory format, and is described in Memory Mapped AXI4 Interface.

  Use the following equation to compute the stride from the width (in pixels) in tile-scan order for a tile size of n × 4 (where n = 32 or 64):

  $$\text{Stride in Bytes} >= (((\text{Width} + (n-1)) \& \sim(n-1)) * 4 * \text{bit-depth}) / 8$$

  ***Note*:**

  . Due to a current limitation of the IP core when configured for samples per clock=8, the minimum stride required for tiled video format with bit-depth=10 needs to be calculated using n=64 in the above formula for both 32x4 and 64x4 tiles.

  . Padding bytes is sometimes necessary (hence, the ≥ in the equation) to make sure that every row of pixels starts at an address that is aligned with the size of the data on the memory mapped interface as described in Stride (0x0020) Register.

- The `Width` value must be a multiple of `Pixels per Clock` as selected in the Vivado IDE for this core.
- For Tile video format, the `Width` value must be a multiple of four and `Pixels per Clock`. `Height` value must be multiple of eight.

Modes of Operation

The Video Frame Buffer Read and Video Frame Buffer Write support two modes of operation.

- **Synchronous Mode:** When an interrupt is triggered, the core interrupt service routine (ISR) checks to confirm if current frame processing is complete, and then calls any additional user programmable callback functions. In the callback function, the register settings for the next frame should be programmed, that is, from where the buffer address should read next.

  For example, upon finishing a frame, the Video Frame Buffer Write IP triggers an interrupt at `ap_done`. At this time, the callback function is run to program a new buffer address. After running the callback function, the core restarts, and the next frame is processed.

- **Asynchronous Mode:** In asynchronous mode, the register settings for the next frame are programmed at any time rather than waiting first for an interrupt.

For example, the Video Frame Buffer Write IP finishes processing a frame and issues an interrupt. At this time, the buffer address is passed to the Video Frame Buffer Read IP, which is in the middle of processing its own frame. The Video Frame Buffer Read buffer address is programmed asynchronously. When the Video Frame Buffer Read is finished processing its current frame, the newly programmed buffer address takes an effect and the Video Frame Buffer Read IP processes the next frame.

Usage

To integrate and use the Video Frame Buffer Read and Video Frame Buffer Write core drivers in the application, perform the following steps:

***Note*:** The Video Frame Buffer Read is used as an example.

1. Include the driver header file `xv_frmbufrd_l2.h` that contains the frame buffer instance object definition.
2. Declare an instance of the frame buffer core type: `XV_FrmbufRd_l2 FrmbufRd.`
3. Initialize the frame buffer instance at power on.

   ```
   int XVFrmbufRd_Initialize(XV_FrmbufRd_l2 *InstancePtr, u16
   DeviceId);
   ```

   This function accesses the hardware configuration and initializes the core to the power on default state.
4. The application must register the ISR with the system interrupt controller and set the application callback function. This function will then be called by the Video Frame Buffer Read or Video Frame Buffer Write driver when the IRQ is triggered.
   - Register the core ISR routine `XVFrmbufRd_InterruptHandler` or `XVFrmbufWr_InterruptHandler` with the system interrupt controller.
   - Register the application callback function that should be called within the interrupt context. This can be done using the API `XVFrmbufRd_SetCallback` or `XVFrmbufWr_SetCallback`.
5. If applicable, write the application-level callback function. An example action to be performed here would be to update the buffer address from where to read the next frame data. This allows the application to render the frame updates in memory and on screen.
6. Write a function to configure the core. The following is an example event sequence that might be performed here.
   a. Set the video properties:

   ```
   void XVFrmbufRd_SetMemFormat(XV_FrmbufRd_l2 *InstancePtr,
   u32 StrideInBytes,
   XVidC_ColorFormat MemFmt
   const XVidC_VideoStream *StrmIn);
   ```

b. Set the frame buffer base address.

```
int XVFrmbufRd_SetBufferAddr(XV_FrmbufRd_l2 *InstancePtr, UINTPTR
Addr);
```

For semi-planar formats, set the buffer base address for the second plane:

```
int XVFrmbufRd_SetChromaBufferAddr(XV_FrmbufRd_l2 *InstancePtr,
UINTPTR Addr);
```

For 3 planar formats, set the buffer base address for the 2nd plane and 3rd plane:

```
int XVFrmbufRd_SetChromaBufferAddr(XV_FrmbufRd_l2 *InstancePtr,
UINTPTR Addr); int XVFrmbufRd_SetVChromaBufferAddr(XV_FrmbufRd_l2
*InstancePtr,UINTPTR Addr);
```

c. Enable the interrupts:

```
void XVFrmbufRd_InterruptEnable(XV_FrmbufRd_l2 *InstancePtr, u32
IrqMask);
```

d. Start the core:

```
void XVFrmbufRd_Start(XV_FrmbufRd_l2 *InstancePtr);
```

Also, to help debug the Video Frame Buffer Read core a Debug API is available that provides the state of the core.

```
void XVFrmbufRd_DbgReportStatus(XV_FrmbufRd_l2 *InstancePtr);
```

***Note*:**

1. Resolution changes are not possible on the fly. If either the master input stream or the output stream resolution must be changed during operation, the application must reset the Video Frame Buffer Read and Video Frame Buffer Write, that is, toggle `ap_rst_n`, and reconfigure the core for the new resolution. After reset, all registers are cleared to 0.
2. When configuring the frame buffer cores, the driver checks to ensure that the frame size, stride, and memory video format are valid. If any of these configurations are invalid, then the action will not be applied and the API will return an error code. The application code should check the return status of all APIs to make sure the required action was completed successfully and, if not, take corrective action.

Appendix D

Debugging

This appendix includes details about resources available on the AMD Support website and debugging tools.

If the IP requires a license key, the key must be verified. The AMD Vivado™ design tools have several license checkpoints for gating licensed IP through the flow. If the license check succeeds, the IP can continue generation. Otherwise, generation halts with an error. License checkpoints are enforced by the following tools:

- Vivado Synthesis
- Vivado Implementation
- write_bitstream (Tcl command)

IMPORTANT! *IP license level is ignored at checkpoints. The test confirms a valid license exists. It does not check IP license level.*

Finding Help with AMD Adaptive Computing Solutions

To help in the design and debug process when using the core, the Support web page contains key resources such as product documentation, release notes, answer records, information about known issues, and links for obtaining further product support. The Community Forums are also available where members can learn, participate, share, and ask questions about AMD Adaptive Computing solutions.

Documentation

This product guide is the main document associated with the core. This guide, along with documentation related to all products that aid in the design process, can be found on the AMD Adaptive Support web page or by using the AMD Adaptive Computing Documentation Navigator. Download the Documentation Navigator from the Downloads page. For more information about this tool and the features available, open the online help after installation.

Answer Records

Answer Records include information about commonly encountered problems, helpful information on how to resolve these problems, and any known issues with an AMD Adaptive Computing product. Answer Records are created and maintained daily to ensure that users have access to the most accurate information available.

Answer Records for this core can be located by using the Search Support box on the main AMD Adaptive Support web page. To maximize your search results, use keywords such as:

- Product name
- Tool message(s)
- Summary of the issue encountered

A filter search is available after results are returned to further target the results.

Master Answer Record for the Core

Master Answer Record for the Video Frame Buffer Read

AR: 68764

Master Answer Record for the Video Frame Buffer Write

AR: 68765

Technical Support

AMD Adaptive Computing provides technical support on the Community Forums for this AMD LogiCORE™ IP product when used as described in the product documentation. AMD Adaptive Computing cannot guarantee timing, functionality, or support if you do any of the following:

- Implement the solution in devices that are not defined in the documentation.
- Customize the solution beyond that allowed in the product documentation.
- Change any section of the design labeled DO NOT MODIFY.

To ask questions, navigate to the Community Forums.

Debug Tools

There are many tools available to address Video Frame Buffer design issues. It is important to know which tools are useful for debugging various situations.

Vivado Design Suite Debug Feature

The AMD Vivado™ Design Suite debug feature inserts logic analyzer and virtual I/O cores directly into your design. The debug feature also allows you to set trigger conditions to capture application and integrated block port signals in hardware. Captured signals can then be analyzed. This feature in the Vivado IDE is used for logic debugging and validation of a design running in AMD devices.

The Vivado logic analyzer is used to interact with the logic debug LogiCORE IP cores, including:

- ILA 2.0 (and later versions)
- VIO 2.0 (and later versions)

See the *Vivado Design Suite User Guide: Programming and Debugging* (UG908).

Hardware Debug

Hardware issues can range from link bring-up to problems seen after hours of testing. This section provides debug steps for common issues. The AMD Vivado™ debug feature is a valuable resource to use in hardware debug. The signal names mentioned in the following individual sections can be probed using the debug feature for debugging the specific problems.

General Checks

Ensure that all the timing constraints for the core were properly incorporated from the example design and that all constraints were met during implementation.

- Does it work in post-place and route timing simulation? If problems are seen in hardware but not in timing simulation, this could indicate a PCB issue. Ensure that all clock sources are active and clean.
- If using MMCMs in the design, ensure that all MMCMs have obtained lock by monitoring the `locked` port.
- If the streaming data coming out of the Video Frame Buffer Read core has the wrong colors, check the Memory Video Format register and ensure that the proper video format used in memory is programmed in the register. Also, each memory video format has a corresponding streaming video format. For example, the memory format RGB8 outputs RGB streaming data. Ensure that the system is expecting the proper streaming video format.
- If the data written to memory by the Video Frame Buffer Write seems to be in the wrong format, check the Memory Video Format register. Ensure that the proper video format used in memory is programmed in the register.

Appendix E

Additional Resources and Legal Notices

Finding Additional Documentation

Technical Information Portal

The AMD Technical Information Portal is an online tool that provides robust search and navigation for documentation using your web browser. To access the Technical Information Portal, go to https://docs.amd.com.

Documentation Navigator

Documentation Navigator (DocNav) is an installed tool that provides access to AMD Adaptive Computing documents, videos, and support resources, which you can filter and search to find information. To open DocNav:

- From the AMD Vivado™ IDE, select **Help → Documentation and Tutorials**.
- On Windows, click the **Start** button and select **Xilinx Design Tools → DocNav**.
- At the Linux command prompt, enter `docnav`.

***Note*:** For more information on DocNav, refer to the *Documentation Navigator User Guide* (UG968).

Design Hubs

AMD Design Hubs provide links to documentation organized by design tasks and other topics, which you can use to learn key concepts and address frequently asked questions. To access the Design Hubs:

- In DocNav, click the **Design Hubs View** tab.
- Go to the Design Hubs web page.

Support Resources

For support resources such as Answers, Documentation, Downloads, and Forums, see Support.

References

These documents provide supplemental material useful with this guide:

1. *Vivado Design Suite: AXI Reference Guide* (UG1037)
2. *AXI4-Stream Video IP and System Design Guide* (UG934)
3. *Vivado Design Suite User Guide: Designing IP Subsystems using IP Integrator* (UG994)
4. *Vivado Design Suite User Guide: Designing with IP* (UG896)
5. *Vivado Design Suite User Guide: Getting Started* (UG910)
6. *Vivado Design Suite User Guide: Logic Simulation* (UG900)
7. *ISE to Vivado Design Suite Migration Guide* (UG911)
8. *Vivado Design Suite User Guide: Programming and Debugging* (UG908)
9. *Vivado Design Suite User Guide: Implementation* (UG904)
10. *Vitis Software Platform Release Notes* (UG1742)
11. *Vitis High-Level Synthesis User Guide* (UG1399)
12. *Versal AI Edge Gen 2 H.264/H.265/JPEG Video Codec Unit 2 Solutions LogiCORE IP Product Guide* (PG447)

Revision History

The following table shows the revision history for this document.

Section	Revision Summary
11/20/2025 Version 3.0	
Tile Format	Updated the section.
Stride (0x0020) Register	Updated the section.
Interface	Updated the section.
General Settings	Updated the section.
User Parameters	Updated the section.
Running the Example Design	Updated the section.

Section	Revision Summary
Prerequisites	Updated the section.
05/29/2025 Version 3.0	
Entire document	• Elaborated the description of Tile formats. • Added URAM support for line buffers. • Updated Vitis application flow as per Vitis Unified IDE.
12/11/2024 Version 3.0	
Entire Document	Added Y12, Y_UV12, Y_UV12_420, Y_U_V12, YUV_444 3 planar 12-bit video formats, tile format, and alpha formats in frame buffer write
11/19/2023 Version 2.5	
Entire Document	Added Y_U_V8_420 and Y_U_V8 video formats where applicable
04/12/2023 Version 2.4	
IP Facts	Updated the PRU links.
05/11/2022 Version 2.4	
Chapter 3: Product Specification	Added Y_U_V10.
User Parameters	Updated the table.
Upgrading in the Vivado Design Suite	Updated the section.
10/27/2021 Version 2.3	
N/A	• Added support for 16K resolution and 3 Planar video format • Updated enhancement for field ID signal
08/09/2021	
Chapter 6: Example Design	Added support for AMD Versal™ example design.
02/04/2021 Version 2.2	
Vitis Software Platform Workflow	Updated for v2.2.
07/08/2020 Version 2.1	
N/A	• Updated AXI-Stream Video Interface section in Chapter 2. • Updated Control (0x0000) Register in Chapter 2. • Updated Stride (0x0020) Register in Chapter 2. • Updated Plane1 Buffer (0x0030) and Plane 2 Buffer (0x003C) Registers in Chapter 2. • Updated Samples Per Clock description in Chapter 4.
12/19/2019 Version 2.1	
Chapter 5: Design Flow Steps	Vitis flow updated.
11/14/2018 Version 2.1	
Features	Added support for 8K30.
fid error	Updated the UG934 link.
Memory Mapped AXI4 Interface	Added Table 10: IP Format to Common Video Pixel Format.
Table 10: IP Format to Common Video Pixel Format	Updated UG934 link in Note.
N/A	Fixed AR 68764 link.

Section	Revision Summary
04/04/2018 Version 2.0	
N/A	• Added support for interlaced video • Added BGR8 video format • Added support for ZCU102, ZCU104, and ZCU106 boards in the example design.
10/04/2017 Version 2.0	
N/A	Added new video formats, added second buffer for semi-planar formats, added support for 64-bit addressing on memory interfaces.
04/05/2017 Version 1.0	
N/A	Initial AMD release.

Please Read: Important Legal Notices

OR DISTRIBUTING ANY SYSTEMS THAT INCORPORATE PRODUCTS, THOROUGHLY TEST SUCH SYSTEMS FOR SAFETY PURPOSES. USE OF PRODUCTS IN A SAFETY APPLICATION WITHOUT A SAFETY DESIGN IS FULLY AT THE RISK OF CUSTOMER, SUBJECT ONLY TO APPLICABLE LAWS AND REGULATIONS GOVERNING LIMITATIONS ON PRODUCT LIABILITY.

Copyright